



**Capital University [Formerly Helwan University]
Faculty of Computing and Artificial Intelligence
Computer Science Department**

**A HYBRID WEB-BASED FRAMEWORK FOR UNIVERSITY COURSE
TIMETABLING PROBLEM USING EVOLUTIONARY ALGORITHMS**

A graduation project dissertation by:

Ahmed Ragab Desouky Mohamed [20220021]

Ahmed Amr Mostafa Fouad [20220029]

Ahmed Mohamed Mohamed Saleh [20220039]

Ahmed Mohamed Mahmoud Ibrahim [20220040]

Abdelrahman Hany Hegazy Saber [20220269]

Asem Hamed Elsayed Mahmoud [20220251]

**Submitted in partial fulfilment of the requirements for the degree of Bachelor of
Science in Computing & Artificial Intelligence at the Computer Science
Department, the Faculty of Computing & Artificial Intelligence, Capital University
[formerly Helwan University]**

Supervised by:

Dr. Amr S. Ghoneim

June 2026

Declaration of Originality

We, the undersigned, hereby declare that the thesis titled students' is our own original work and has not been submitted, in whole or in part, for a degree or other qualification at any other university or institution.

We further confirm that:

- **All sources of information, including books, journal articles, websites, and any other external materials, have been duly acknowledged and properly cited in accordance with the university's academic reference guidelines.**

Student Name 1: Ahmed Mohamed Mohamed Saleh

Signature:

Date:

Student Name 2: Ahmed Mohamed Mahmoud Ibrahim

Signature:

Date:

Student Name 3: Ahmed Ragab Desouki Mohamed

Signature:

Date:

Student Name 4: Ahmed Amr Moustafa Fouad

Signature:

Date:

Student Name 5: Asem Hamed Elsied Mahmoud

Signature:

Date:

Student Name 6: Abdelrahman Hany Hegazy Saber

Signature:

Date:

Abstract

The University Course Timetabling Problem (UCTP) is a classically complex, NP-hard challenge that demands robust computational strategies to satisfy strict academic constraints while optimizing resource allocation. This thesis proposes a novel, high-performance, web-based educational management framework that integrates advanced metaheuristic solvers specifically Memetic Algorithms (MA), Hybrid Approaches (HGATS), Guided Search NSGA-II (GSNSGA-II), and Cultural Algorithms (CA) into a unified, user-centric environment.

The mathematical formulation of constraints and the underlying algorithmic logic are grounded in foundational operational research. To address the inherent computational intensity of evolutionary optimization, the system employs a decoupled, asynchronous architecture. The core algorithmic engine is implemented in C++ to ensure maximum execution speed, orchestrated via a Python FastAPI backend, and managed through a Redis-based task queue. This infrastructure allows for the seamless, responsive execution of complex scheduling tasks within a modern Next.js and React user interface.

Experimental evaluations were conducted using internationally recognized benchmarks from the International Timetabling Competition (ITC-2002 and ITC-2007). The results demonstrate that the implemented GSNSGA-II provides exceptional robustness, consistently achieving 100% feasibility in hard-constraint satisfaction across diverse datasets. Furthermore, the integration of Cultural Algorithms, which utilize a dual-inheritance belief space to prune the search process, proved highly effective in minimizing soft-constraint violations in large-scale instances. The Memetic Algorithm implementations also demonstrated the capability to reach optimal, zero-violation schedules in specific test cases.

In conclusion, this research successfully bridges the gap between theoretical evolutionary optimization and practical administrative software engineering. By providing a scalable, web-based platform that abstracts the complexity of metaheuristic solvers from the end-user, this system serves as a viable model for modern institutional timetabling. Future work will focus on expanding constraint modeling to include lecturer-specific matrices and implementing distributed processing to scale the optimization engine for comprehensive, university-wide datasets.

Acknowledgment

We would like to express our deepest gratitude to our supervisor, Dr. Amr S. Ghoneim, for his invaluable guidance, patience, and encouragement throughout the course of this research. His expertise and critical feedback were instrumental in shaping the methodology and ensuring the academic rigor of this work.

We are also deeply grateful to the Faculty of Computers and Artificial Intelligence at Capital University for providing the academic environment and resources necessary to conduct this study. We extend our thanks to the faculty members who supported our educational journey and provided the foundational knowledge required for this project.

Special thanks are due to our colleagues and lab mates for their technical support, helpful discussions, and for fostering a collaborative environment that made the challenges of this project more manageable.

Finally, we owe our heartfelt thanks to our families and friends. Their constant encouragement, sacrifices, and belief in our potential have been the source of our strength and motivation. Without their unwavering support, the completion of this thesis would not have been possible.

Contents

Declaration of Originality	2
Abstract	3
Acknowledgment	4
Contents	4
List of Figures	8
List of Tables	8
List of Algorithms	8
List of Abbreviations and Acronyms	8
Chapter 1: Introduction and Background	9

Scheduling vs. Timetabling	9
Educational Timetabling Problems	9
Constraint Modeling and Solution Quality	10
Hard Constraints	10
Soft Constraints	10
Evaluating Solution Quality	11
Variants of UCTP	11
Problem Statement: The Computational Challenge of UCTPs	11
Project Scope and Limitations	12
In-Scope Objectives	12
Out of Scope (Project Boundaries)	12
Chapter 2: Problem Definition	13
Introduction	13
Core Mathematical Notation	13
Formulation of the PECTP	15
Formulation of MOUCTP	16
Benchmark Dataset	16
Chapter 3: The Proposed Memetic Algorithm (MA)	17
Neighbourhoods	17
Neighbourhood Structure N1: Single Event Displacement	17
Neighbourhood Structure N2: Pairwise Event Swap	17
Neighbourhood Structure N3: 3-Cycle Permutation	17
Matching Algorithm	17
Graph-Theoretic Construction	18
Transformation to Network Flow	18
The Critical Deterministic Fallback Mechanism	19
Local Search Strategies	20
Local Search 1	20
Local Search 2	21
Steady State Genetic Algorithm	22
Memetic Algorithm	23
Selection	23

Crossover	24
Mutation	24
Evaluation	24
Algorithmic Enhancements and Architectural Optimizations	25
Chapter 4: Culture Algorithm	26
Introduction	26
Proposed Approach	27
Evaluation	29
Results on Small Instances	30
Results on Medium Instances	30
Chapter 5: Multi-Objective UCTP	30
Introduction	30
Matching Algorithm	31
Graph-Theoretic Construction	31
Transformation to Network Flow	31
The Critical Deterministic Fallback Mechanism	32
Local Search Strategies	33
Local Search 2	33
Local Search 5	34
The GS Strategy	36
Data Structures MEM _i (i = 1, 2, 3)	36
Child Generation Process	39
Genetic Operators	40
Objective Functions and Constraints Handling	40
Selection Mechanism	41
Crossover Operator	41
Mutation Operator	42
Guided Search NSGA-II (GSNSGA)	42
Evaluation	46
Results on Small Instances	46
Results on Medium Instances	47
Chapter 6: Hybrid Genetic Algorithm with Tabu Search (HGATS)	47

Introduction	47
The HGATS Framework	48
Phase I: Guided Search Genetic Algorithm (GSGA)	49
Population Initialization	49
Fitness Evaluation	49
Guided Search and MEM Structure	49
Crossover and Mutation Operators	50
Crossover Operator	50
Mutation Operator	50
Local Search Improvement	50
Phase II: Tabu Search	51
Neighborhood Generation	51
Tabu List Management	51
Solution Improvement	52
HGATS Algorithm	52
Evaluation	53
Results	53
Chapter Summary	54
Chapter 7: System Architecture	54
Functional Requirements	54
Admin:	54
Student:	55
Instructor:	56
Non-functional requirements	56
Use-Case Diagram	58
System Design	59
ERD	60
Activity Diagrams	61
Admin	61
Students	62
Chapter 8: Conclusion and Future Work	62
Conclusion	62

Future Work	63
Algorithmic and Constraint Expansion	63
Architectural and System Scaling	64

List of Figures

Figure 1: Memetic Convergence Chart.....	25
--	----

List of Tables

Table 1: Features of UCTP Instances	16
Table 2: ITC-2007 Instances	16
Table 3: Memetic Evaluation	24
Table 4: Cultural algorithm evaluation on small instances.....	30
Table 5: Cultural algorithm evaluation on medium instances	30
Table 6: GSNASGA evaluation on small instances.....	47
Table 7: GSNASGA evaluation on medium instances.....	47
Table 8: HGATS Results	53

List of Algorithms

1. Steady State Genetic Algorithm (SSGA)
2. The Proposed Memetic Algorithm (MA)
3. Local Search Strategy 1 (LS1)
4. Local Search Strategy 2 (LS2)
5. Proposed Hybrid Approach (HGATS)
6. Guided Search NSGA-II (GSNSGA)
7. ConstructMEM3()
8. GuidedSearch3()
9. Local Search Strategy 4 (LS4)
10. Local Search Strategy 5 (LS5)
11. Tabu Search
12. Guided Search Genetic Algorithm (GSGA)

List of Abbreviations and Acronyms

<i>Acronym</i>	<i>Full Meaning</i>
----------------	---------------------

<i>EA</i>	Evolutionary Algorithm
<i>GA</i>	Genetic Algorithm
<i>GS</i>	Guided Search
<i>GSNSGA</i>	Guided Search Non-dominated Sorting Genetic Algorithm
<i>HGATS</i>	Hybrid Genetic Algorithm Tabu Search Approach
<i>ITC-2002</i>	the 2002 International Timetabling Competition
<i>ITC-2007</i>	the 2007 International Timetabling Competition
<i>LS</i>	Local Search
<i>MA</i>	Memetic Algorithm
<i>UCTP</i>	University Course Timetabling Problem
<i>PECTP</i>	Post Enrollment Course Timetabling Problem
<i>MOUCTP</i>	Multi-Objective University Course Timetabling Problem
<i>NSGA-II</i>	Elitist Non-dominated Sorting Genetic Algorithm
<i>SSGA</i>	Steady State Genetic Algorithm

Chapter 1: Introduction and Background

Scheduling vs. Timetabling

While the terms "timetabling" and "scheduling" are frequently used interchangeably in literature, they represent distinct operational research challenges. According to Wren, scheduling is defined as the allocation of resources to objects placed in space-time, subject to constraints, to minimize the total cost of the resources used. This broad definition encompasses the allocation of materials, labor, and equipment to tasks over time, such as in production or employee shift scheduling.

Conversely, Wren defines timetabling as the allocation of given resources to objects placed in space and time, subject to constraints, to satisfy a set of desirable objectives as nearly as possible. Therefore, timetabling problems primarily focus on ensuring adequate resources (personnel, facilities, equipment) are available for predefined events, navigating complex constraints common in educational, transport, or sporting domains.

Educational Timetabling Problems

An educational timetabling problem involves assigning a series of events such as lectures, examinations, or meetings to a limited number of resources while satisfying a

strict set of operational rules. Schaerf classifies educational timetabling into three primary categories, each possessing unique constraints and requirements:

- **School Timetabling:** The weekly scheduling of all classes within a school, strictly avoiding scenarios where a teacher is assigned to two classes simultaneously, or multiple classes are assigned to a single teacher.
- **Examination Timetabling:** As defined by Carter and Laporte, this involves assigning examinations to a limited number of available time periods and rooms to ensure students do not experience scheduling conflicts or clashes.
- **University Course Timetabling:** Also defined by Carter and Laporte, the University Course Timetabling Problem (UCTP) is a multi-dimensional assignment challenge. It involves assigning students and faculty to specific courses or events and subsequently mapping those events to appropriate classrooms and timeslots without conflict, while ensuring a conducive learning environment.

This thesis focuses specifically on the University Course Timetabling Problem (UCTP).

Constraint Modeling and Solution Quality

UCTPs belong to a class of highly constrained, hard-to-solve combinatorial optimization problems. The parameters defining a valid timetable are divided into two distinct categories: hard constraints and soft constraints.

Hard Constraints

Hard constraints are mandatory conditions that must be completely satisfied to generate a viable schedule. A timetable cannot be implemented if any hard constraint is violated. Standard UCTP hard constraints include:

- Lectures sharing common students cannot be scheduled concurrently.
- Lectures must be assigned to rooms that meet specific facility requirements and seating capacities.
- No two independent lectures can occupy the same room at the same time.
- A faculty member cannot be assigned to multiple events simultaneously.
- The schedule must respect the maximum allowable number of teaching periods per day.

Soft Constraints

Soft constraints represent highly desirable conditions that improve the overall quality of the timetable. While violations are permitted, the optimization objective is to minimize them. Standard soft constraints include:

- Aligning the schedule with a faculty member's preferred availability or minimizing their travel time between campus locations.
- Respecting minimum and maximum weekly workload limits for faculty.

- Dispersing lectures for a specific subject evenly across the week to optimize student learning retention.
- Preventing students or faculty from attending more than two consecutive lectures without a break.
- Avoiding isolated schedules where a student has only a single lecture on a given day.
- Preserving specific timeslots for extracurricular or non-academic activities.

Evaluating Solution Quality

Based on the satisfaction of these constraints, the quality of an algorithmic output is classified into three tiers:

1. **Infeasible Solution:** A schedule that fails to satisfy one or more hard constraints.
2. **Feasible Solution:** A schedule that successfully satisfies 100% of the hard constraints, regardless of the number of soft constraint violations.
3. **Optimal Solution:** A theoretical ideal where a feasible schedule simultaneously achieves zero soft constraint violations.

Variants of UCTP

As formalized by the International Timetabling Competition (ITC), UCTPs are generally divided into two distinct tracks based on institutional requirements:

- **Curriculum-Based Course Timetabling (CBCTP):** Conflicts between courses are determined strictly by the published university curricula, regardless of individual student choices.
- **Post Enrolment-Based Course Timetabling (PECTP):** Also known as event timetabling, this approach constructs the schedule *after* student enrollment data is collected, ensuring all students can attend their individually chosen courses without clashes. This introduces highly complex, dynamic hard constraints that closely mirror real-world institutional challenges.

This thesis exclusively addresses the Post Enrolment-Based Course Timetabling Problem (PECTP).

Problem Statement: The Computational Challenge of UCTPs

Despite extensive research, UCTPs remain inherently difficult to solve for two primary reasons.

First, there is no universally standardized definition of the problem. Different academic institutions possess highly idiosyncratic constraints; a rigid hard constraint at one university may be irrelevant or directly opposed to the operational rules of another. This variance makes formulating a generalized mathematical model exceptionally difficult.

Second, almost all variants of the UCTP are classified as NP-complete. Unlike P-problems, which can be solved by a deterministic algorithm in polynomial time, NP-complete problems possess a solution space that scales exponentially with the input size, rendering exhaustive computational search impossible.

To illustrate this complexity, consider a university attempting to schedule 1,500 events across 45 timeslots and 100 rooms. The theoretical solution space encompasses $(45 \times 100)^{1500}$ possible combinations. Because the vast majority of these combinations represent infeasible schedules, finding an optimal, or even feasible, solution within a reasonable timeframe cannot be achieved through standard deterministic methods. Consequently, there is a critical need for robust, advanced metaheuristic frameworks capable of intelligently navigating this massive search space to automate the generation of highly constrained institutional timetables.

Project Scope and Limitations

To ensure the feasibility, academic rigor, and timely completion of this research, clear boundaries have been established regarding the system's capabilities and target domain.

In-Scope Objectives

This project is strictly focused on designing, implementing, and evaluating a robust solution for the Post Enrolment-Based Course Timetabling Problem (PECTP). The scope encompasses:

- The mathematical modeling of standard hard and soft constraints defining the PECTP.
- The development of a high-performance, C++ based algorithmic engine utilizing specific metaheuristics (MA, HGATS, GSNSGA-II, and CA).
- The implementation of a decoupled web architecture utilizing a Next.js frontend, a FastAPI backend, and a Redis task queue.
- The evaluation of the implemented algorithms using static, internationally recognized benchmark datasets (ITC-2002 and ITC-2007).

Out of Scope (Project Boundaries)

To maintain a realistic understanding of constraints and focus on the core algorithmic optimization, the following elements are explicitly excluded from the current scope of this thesis:

- **Curriculum-Based Course Timetabling (CBCTP):** The system is not designed to solve CBCTP tracks. The architectural and algorithmic models assume that student enrollment data is the primary driver for conflict resolution, rather than predefined university curricula.
- **Live Institutional ERP Integration:** While the framework is designed as a web-based educational management tool, it is built as a standalone prototype. Direct

integration with existing, live university databases or Enterprise Resource Planning (ERP) systems is outside the scope of this project.

- **Explicit Lecturer Entity Modeling:** To maintain strict methodological alignment with the standardized evaluation metrics of the ITC-2002 and ITC-2007 benchmark datasets, this system structurally abstracts the "lecturer" entity. Within these globally recognized datasets, scheduling conflicts are modeled implicitly via mathematical event-to-event conflict matrices (e.g., establishing a hard constraint that Event A and Event B cannot occur concurrently) rather than explicitly tracking individual professor profiles. Consequently, the integration of distinct lecturer database entities, individualized faculty availability matrices, and personalized teaching preferences is deliberately excluded from the current prototype to ensure the algorithm's performance is measured purely against established baseline standards.

Chapter 2: Problem Definition

Introduction

In a UCTP, we assign an event (course, lecture) into a time slot and also assign a number of resources (students and rooms) in such a way that there is no conflict between the rooms, time slots, and events.

The goal is to find a schedule that satisfies all the hard constraints and minimizes the violation of soft constraints.

Feasibility is defined as the satisfaction of all hard constraints, while optimality is defined as the minimization of the total penalty incurred by the violation of soft constraints.

The feasible solution is a schedule that satisfies all hard constraints, while the optimal solution is a schedule that has zero or minimal violation of soft constraints.

Core Mathematical Notation

we deal with the following hard constraints:

- H1: No student attends more than one event at the same time.
- H2: The room is big enough for all the attending students and satisfies all the features required by the event.
- H3: Only one event is in a room at any time slot.

and the following soft constraints:

- S1: No student should be required to attend an event in the last time slot of a day.
- S2: No student should attend more than two events in a row.
- S3: No student should have a single event in a day.

the problem can be formally defined as follows:

Given a set of n events (classes, subjects) $E = e_1, e_2, \dots, e_n$ to be scheduled in a set of 45 timeslots $T = t_1, t_2, \dots, t_{45}$ nine for each day in a five-day week, a set of m available rooms $R = r_1, r_2, \dots, r_m$ in which events can take place, a set of k number of students $S = s_1, s_2, \dots, s_k$ who attend the events and a set of l available features $F = f_1, f_2, \dots, f_l$ that are satisfied by rooms and required by each event.

In addition, interrelationships between these sets are given by five matrices described as follows:

- The first matrix $A_{k,n}$, called the Student-Event matrix, shows which event is attended by which students. In $A_{k,n}$, the value of $a_{i,j}$ is 1 if student $a_{i,j} \in S$ should attend event $i \in E$ otherwise, the value is 0.
- The second matrix $B_{n,n}$, called the Event-Conflict matrix, indicates whether two events can be scheduled in the same time slot or not.
- The third matrix $C_{m,l}$, called the Room-Features matrix, gives the features that each room possesses, where the value of a cell $c_{i,j}$ is 1 if $i \in R$ has a feature $j \in F$ otherwise, the value is 0.
- The fourth matrix $D_{n,l}$, called the Event-Features matrix, gives the features required by each event. It means that event $i \in E$ needs feature $j \in F$ if and only if $d_{i,j} = 1$.
- The last matrix $G_{n,m}$, called the Event-Room matrix, lists the possible rooms to which each event can be assigned. Through this matrix, we can quickly identify all rooms that are suitable in size and feature for each event.

Usually, a matrix is used for assigning each event to a room r_i and a time slot t_i .

Each pair of (r_i, t_i) is assigned a particular number which corresponds to an event. If a room r_i in a time slot t_i is free, or no event is placed, then "-1" is assigned to that pair. In this way, we assure that there will be no more than one event assigned to the same pair so that one of the hard constraints will always be satisfied.

For the room assignment, we used a matching algorithm described by Rossi Doria. For every timeslot, there is a list of events taking place in it and a preprocessed list of possible rooms to which the placement of events can be occurred. The matching

algorithm uses a deterministic network flow algorithm and gives the maximum cardinality matching between rooms and events

In general, the solution to a UCTP can be represented in the form of an ordered list of pairs (r_i, t_i) , of which the index of each pair is the identification number of an event $e_i \in E$ ($i = 1, 2, \dots, n$).

The goal of the UCTP is to minimize the soft constraint violations of a feasible solution (a feasible solution means that no hard constraint violation exists in the solution). The objective function $f(s)$ for a timetable s is the weighted sum of the number of hard constraint violations $\#hcv$ and soft constraint violations $\#scv$, as shown in the following formula: $f(s) = C \times \#hcv + \#scv$.

Equation 1: Objective Function

Where C is a constant, which is larger than the maximum possible number of soft constraint violations. In all the experiments carried out in later chapters for the UCTP, we set $C = 10^6$.

Formulation of the PECTP

we deal with the following hard constraints:

- H1: No student attends more than one event at the same time.
- H2: The room is big enough for all the attending students and satisfies all the features required by the event.
- H3: Only one event is in a room at any time slot.
- H4: Events should only be assigned to time slots that are predefined as available for those events.
- H5: Where specified, events should be scheduled to occur in the correct order.

and the following soft constraints:

- S1: No student should be required to attend an event in the last time slot of a day.
- S2: No student should attend more than two events in a row.
- S3: No student should have a single event in a day.

And extra matrices:

- $H_{n,t}$ is called the Event-Availability matrix, where the value of a cell $H_{i,j}$ is 1 if $e_i \in E$ should take place at time slot $t_j \in T$ otherwise, the value is 0.
- $I_{n,n}$ is called the Event-Preference matrix, where the value of cell $I_{i,j}$ is 1 if $e_i \in E$ must be scheduled before $e_j \in E$, or -1 if $e_i \in E$ has to be placed in the timetable

after $e_j \in E$ otherwise, the value is 0 if there is no restriction of precedence between them.

Formulation of MOUCTP

We will change the objective function from scalar to multi-objective as three objectives $f_1(x)$, $f_2(x)$, $f_3(x)$ respectively, where x is a solution to the MOUCTP.

Benchmark Dataset

Table 1: Features of UCTP Instances

Class	Small	Medium	Large
Number of events	100	400	400
Number of rooms	5	10	10
Number of features	5	5	10
Per room approximate features	3	3	5
Percentage (%) of features used	70	80	90
Number of students	80	200	400
Maximum events per student	20	20	20
Maximum students per event	20	50	100

We need to schedule 100-400 events/courses into 45 timeslots, corresponding to 5 days of 9 hours each day. By assigning events, we have to satisfy room constraints and student constraints. These eleven problem instances are divided into three groups: five small instances, five medium instances, and one large instance. In the experimental studies to be carried out in the following chapters on this dataset, we use “S1” to “S5” to represent small instance 1 to small instance 5, respectively, “M1” to “M5” to represent medium instance 1 to medium instance 5, respectively, and “L” to represent the large instance.

Table 2: ITC-2007 Instances

ITC-2007 Instance	1	2	3	4	5	6	7	8	9	10	11	12
Number of events	400	400	200	200	400	400	200	200	400	400	200	200
Number of rooms	10	10	20	20	20	20	20	20	10	10	10	10
Number of features	10	10	10	10	20	20	20	20	20	20	10	10
Number of students	500	500	1000	1000	300	300	500	500	500	500	1000	1000
Max students per event	33	32	98	82	19	20	43	39	34	32	88	81
Max events per student	25	24	15	15	23	24	15	15	24	23	15	15
Mean features per room	3	4	3	3	2	3	5	4	3	3	3	4
Mean features per event	1	2	2	2	1	2	3	3	1	2	1	23
ITC-2007 Instance	13	14	15	16	17	18	19	20	21	22	23	24
Number of events	400	400	200	200	100	200	300	400	500	600	400	400
Number of rooms	20	20	10	10	10	10	10	10	20	20	20	20
Number of features	10	10	20	20	10	10	10	10	20	20	30	30
Number of students	300	300	500	500	500	500	1000	1000	300	500	1000	1000
Max students per event	20	20	41	40	195	65	55	40	16	22	69	41
Max events per student	24	24	15	15	23	23	14	15	23	25	24	15
Mean features per room	2	3	2	5	4	4	3	3	3	3	5	5
Mean features per event	1	1	3	3	2	2	1	1	1	2	3	3

Chapter 3: The Proposed Memetic Algorithm (MA)

Neighbourhoods

Local search methodologies function by traversing from a current solution state x to a neighboring candidate solution x' generated by a predefined operator.

In the framework proposed by the referenced authors, the local search follows a strict hierarchy using three neighbourhood structures: N_1, N_2, N_3 .

Neighbourhood Structure N1: Single Event Displacement

Definition: Move exactly one event from its current timeslot to a different timeslot.

The N_1 operator is the most fundamental and frequently executed mechanism in the optimization framework. It selects an event e_a in timeslot t_x and attempts to reassign it to another timeslot t_y where $t_x \neq t_y$.

Although N_1 operator is relatively inexpensive, with a search-space complexity of $O(|E| \times |T|)$, it can show diminishing returns when the timetable approaches capacity.

Neighbourhood Structure N2: Pairwise Event Swap

Definition: Swap the timeslots of two distinct events.

When N_1 fails to produce an acceptable move, N_2 is invoked. It selects event e_a in timeslot t_x and event e_b in timeslot t_y , then exchanges their timeslots.

The search-space complexity of N_2 scales as $O(|E|^2)$.

Neighbourhood Structure N3: 3-Cycle Permutation

Definition: Permute three events in three distinct timeslots using one of the two non-trivial 3-cycles.

The N_3 operator selects three events e_a, e_b, e_c in distinct timeslots t_x, t_y, t_z and applies one of two alternative permutations:

Permutation A: $e_a \rightarrow t_y, e_b \rightarrow t_z, e_c \rightarrow t_x$

Permutation B: $e_a \rightarrow t_z, e_b \rightarrow t_x, e_c \rightarrow t_y$

The rationale for N_3 is escaping deep local optima formed by cyclic dependencies that cannot be resolved by isolated N_1 or N_2 moves without temporarily violating hard constraints. Because of its cost, with a search-space complexity of $O(|E|^3)$.

Matching Algorithm

We treat this as a classic Maximum Bipartite Matching problem. We have two disjoint sets: Events (E) and Rooms (R). Edges only exist between an Event and a Room if the

room is suitable (e.g., right size, right equipment). The network flow algorithm finds the largest possible number of valid pairs.

The algorithmic mandate is to assign each overlapping event in E_{t_j} to a distinct room in R such that the capacity and feature requirements are unconditionally satisfied. This assignment puzzle maps perfectly onto the classical Maximum Cardinality Bipartite Matching problem in graph theory. The algorithm must discover a matching configuration that successfully places the absolute maximum number of events into valid rooms. Rossi-Doria et al. explicitly document that this resolution was achieved utilizing a "deterministic network flow algorithm".

Graph-Theoretic Construction

To initiate the matching for a specific timeslot t_j , an unweighted bipartite graph $G = (U, V, E)$ is constructed dynamically in memory. The components of the graph are established as follows:

- Partition U : Represents the set of events scheduled in the current timeslot ($U = E_{t_j}$)
- Partition V : Represents the set of all available rooms ($V = R$)
- Edges E : An unweighted edge (u, v) is instantiated between event node u and room node v if and only if room v fully satisfies the physical requirements of event u . Specifically, the seating capacity of room v must be strictly greater than or equal to the total number of students enrolled in event u , and the feature set of room v must be a complete superset of the feature set required by event u .

Transformation to Network Flow

The identification of a maximum cardinality matching within a bipartite graph can be efficiently resolved by transforming the structure into a maximum network flow problem, which leverages the Ford-Fulkerson theorem. A directed flow network $G' = (U \cup V \cup \{s, t\}, E')$ is generated by introducing two artificial nodes: a super-source node s and a super-sink node t .

The edge configuration is subsequently modified:

- Directed edges with an absolute capacity of 1 are established from the super-source s to every event node $u \in U$.
- Directed edges with an absolute capacity of 1 are established from every room node $v \in V$ to the super-sink t .
- Every original undirected valid connection edge $(u, v) \in E$ is converted into a directed edge flowing from u to v also constrained with a capacity of 1.

According to the classical Max-Flow Min-Cut theorem, the integral maximum flow of units pushed from s to t through the directed network G' corresponds exactly to the maximum cardinality matching in the original bipartite graph G . The deterministic network flow algorithm iterates, pushing flow through augmenting paths within this network until no further unutilized paths can be discovered.

The Critical Deterministic Fallback Mechanism

While the maximum cardinality matching guarantees the optimal assignment of events to valid rooms, instances arise where $|M| < |E_{tj}|$. This scenario occurs frequently in heavily constrained data instances, or during the early, highly stochastic exploratory phases of a metaheuristic where the temporal timeslot assignment might be disastrously suboptimal. In these instances, events are left unplaced because there are simply no remaining valid rooms, or because the metaheuristic has scheduled far too many events in a single timeslot.

If an algorithm leaves events unassigned, the timetable is incomplete, making objective fitness evaluation impossible. To resolve this, Rossi-Doria et al. implemented a strict, uncompromising deterministic fallback protocol to handle overflow events. This ensured that a fully defined timetable, albeit a highly illegal one containing severe hard constraint violations, was always produced for evaluation.

The fallback procedure is explicitly documented in the literature:

"If there are still unplaced events left, it takes them in label order and puts each one into the room time slots and rooms to all the events."

This tie-breaking rule was paramount to the success of the comparative study. Because trajectory-based metaheuristics like Tabu Search and Simulated Annealing rely on deterministic fitness evaluations to map the contours of the search space, the matching algorithm could not rely on stochastic or randomized assignment for overflow events. By systematically forcing unplaced events into valid rooms in label order, the algorithm artificially violated the "one event per room" constraint, thereby generating a measurable, predictable penalty. This continuity maintained a smooth fitness gradient that the metaheuristics could effectively optimize against.

Local Search Strategies

Local Search 1

```
1: input : Individual I selected from the population
2: while Termination condition not reached do
3:   for  $i = 1$  to the total number of events do
4:     if event  $i$  is infeasible then
5:       if there is untried move left then
6:         calculate the next move (first in  $N1$ , then in  $N2$ , and finally in  $N3$ )
7:         apply the matching algorithm to the time slots affected by the move and
           delta-evaluate the result.
8:         if the move reduces hard constraint violation then
9:           make the move
10:        end if
11:      end if
12:    end if
13:  end for
14:  if any hard constraint violations remain then
15:    terminate LS1
16:  else
17:    for  $i = 1$  to total number of events do
18:      if event  $i$  has soft constraint violation then
19:        if there is untried move left then
20:          calculate the next move (first in  $N1$ , then in  $N2$ , and finally in  $N3$ )
21:          apply the matching algorithm to the time slots affected by the move and
            delta-evaluate the result
22:          if the move reduces soft constraints violation then
23:            make the move
24:          end if
25:        end if
26:      end if
27:    end for
28:  end if
29: end while
30: output : A possibly improved individual I
```

LS1 operates in two steps.

In the first step (lines 3-13 in Algorithm 8), LS1 checks hard-constraint violations for each event while ignoring soft-constraint violations. If an event has hard-constraint violations, LS1 tries to resolve them by applying moves in neighbourhoods $N1$, $N2$, and $N3$ in order until a termination condition is reached, for example:

- improvement is found, or
- the maximum number of steps s_{max} is reached.

The value of s_{max} is set differently for different problem instances. After each move, the matching algorithm is applied to the affected time slots to resolve room-allocation disturbance, and the move is delta-evaluated (hard and soft violations before and after the move).

If there is no untried move left in the neighbourhood for an event, LS1 continues to the next event. After applying neighbourhood moves to each event, if any hard-constraint violation remains, LS1 stops. Otherwise, LS1 performs the second step (lines 17-27 in Algorithm 8).

In the second step, after reaching a feasible solution, LS1 deals with soft constraints and performs a process similar to the first step on each event to reduce soft-constraint violations. For each event, LS1 tries moves in N1, N2, and N3 in order without violating hard constraints. For each move, the matching algorithm allocates rooms to affected events and the move is delta-evaluated.

When LS1 finishes, we obtain a possibly improved and feasible individual. LS2 is then applied to this individual.

Local Search 2

```

1: input : Individual I after LS1 is applied
2:  $S :=$  randomly select a  $tp$  percentage of time slots from the total time slots of  $T$ 
3: for each time slot  $t_i \in S$  do
4:   for each event  $j$  in time slot  $t_i$  do
5:     calculate the penalty value of event  $j$ 
6:   end for
7:   sum the total penalty value of events in time slot  $t_i$ 
8: end for
9: select the time slot  $w_t$  with the biggest penalty value from  $S$ 
10: for each event  $i$  in  $w_t$  do
11:   calculate a move of event  $i$  in the neighbourhood structure  $N1$ 
12:   apply the matching algorithm to the time slots affected by the move
13:   compute the penalty of event  $i$  and delta-evaluate the result
14: end for
15: if all the moves together reduce hard or soft constraint violations then
16:   apply the moves
17: else
18:   delete the moves
19: end if
20: output : A possibly improved individual I

```

LS2 first randomly selects a tp percentage of time slots from the total set of time slots T . Rather than selecting the worst time slot from all time slots, LS2 randomly selects a subset and then picks the worst one within that subset. This reduces computation time, because penalty values must be calculated for examined time slots.

Then LS2 computes the penalty of each selected time slot and chooses the worst time slot w_t (the one with the largest penalty) for LS operations. In this way, LS2 aims to improve the result produced by LS1.

After selecting w_t , LS2 tries an N1 move for each event in w_t , and checks each event's penalty value before and after the move. If all moves in w_t together reduce hard and/or soft constraint violations, LS2 applies all moves; otherwise, it discards them.

This allows LS2 not only to examine the worst time slot but also to reduce penalties for some events by moving them to other time slots. In general, LS2 can enhance

individuals in the population and improve feasible timetable quality by reducing constraint violations.

Steady State Genetic Algorithm

- 1: randomly initialise a population of solutions
- 2: evaluate the individuals in the population
- 3: **while** the termination condition is not reached **do**
- 4: select two parents through a selection scheme
- 5: *crossover the parents to create a child with a probability P_c*
- 6: *apply mutation to the child with a probability P_m*
- 7: replace the worst member of the population by the child
- 8: **end while**

In direct structural opposition, the Steady-State Genetic Algorithm (SSGA) represents a continuous, overlapping population model. Rather than executing a mass extinction event at the conclusion of each iteration, the SSGA selects only a minuscule subset of the population frequently strictly limited to one or two individuals to undergo recombination and mutation processes. The resulting offspring are then evaluated and injected back into the existing population structure, replacing only the absolute worst-performing individuals while leaving the rest of the genetic pool entirely undisturbed.

The SSGA architecture provides profound operational advantages for complex combinatorial optimization problems such as the UCTP: Firstly, by replacing only a microscopic fraction of the individuals in each generation, the SSGA maintains a highly stable, genetically diverse population reservoir compared to the standard GGA. This deliberate stability approach actively helps preserve genetic diversity and provides a robust safeguard against premature convergence, a fatal evolutionary scenario where a single dominant yet globally suboptimal individual rapidly clones its genetic traits across the entire population matrix, halting productive search operations. Secondly, the computational cost per algorithmic iteration in an SSGA is drastically lower than a GGA. Because only a few new individuals are synthesized at a time, the computationally expensive fitness evaluation function which in the UCTP involves calculating thousands of potential student conflicts and analyzing daily class sequences for soft constraint penalties is executed only once or twice per iterative cycle. While the SSGA generally requires a larger absolute number of generations to mathematically converge on a solution, the aggregate wall-clock time is vastly superior due to the minimal cost of each granular iteration. Thirdly, the SSGA intrinsically operates as an elitist algorithm. Because replacement strictly targets the lowest-ranked members of the population array, the absolute best candidate solutions are virtually immune to destruction. This elite preservation ensures that the algorithm's global best score strictly and monotonically decreases (in a penalty minimization context) over time, mathematically precluding generational regression.

Memetic Algorithm

```
1: input : A problem instance I
2: for  $i = 1$  to population size do
3:   construct random initial solution  $s_i$ .
4:   apply LS1 to the solution  $s_i$ 
5:   apply LS2 to the solution  $s_i$ 
6: end for
7: sort population by fitness
8: while termination condition not reached do
9:   select two parents through a selection scheme
10:  crossover the parents to create a child  $s$  with a probability  $P_c$ 
11:  apply mutation to the child  $s$  with a probability  $P_m$ 
12:  apply LS1 to the child  $s$ 
13:  apply LS2 to the child  $s$ 
14:  child solution  $s$  replaces the worst member of the population
15:  sort population by fitness
16:  found best solution  $s_{best}$  in the population
17: end while
18: output : The best solution  $s_{best}$  achieved for I
```

we present our proposed MA that integrates some new LS approaches into the conventional SSGA to enhance the searching ability of the SSGA for the UCTP.

In the proposed MA, we first initialize the population by randomly creating each individual, via assigning a random time slot for each event according to a uniform distribution and applying the matching algorithm to allocate rooms for events. Two LS strategies, LS strategy 1 (LS1) and LS strategy 2 (LS2) are then applied in order to each member of the population.

LS1 uses three neighbourhood structures, denoted as N1, N2, and N3, to move events to timeslots and then uses the matching algorithm to allocate rooms to events and timeslots. With LS2, we take the time slot with the worst penalty value from a set of randomly selected time slots and seek to improve it by trying to move each event in that time slot to another one in the neighbourhood structure N1 and then using the matching algorithm for room allocations for those involved events.

After the initialization of the population, we use the SSGA model where only one child solution is generated with selection, crossover, and mutation at each generation. The child is then improved by LS1 and LS2. In the end, the worst population member is replaced with the new child individual. The iteration continues until one termination condition is reached, e.g., a present time limit t_{max} is reached.

Selection

The binary tournament selection is used, where two parents are randomly selected from the population and the fitter one is used as a parent. At each generation, the tournament selection is applied twice to select two parents for reproduction

Crossover

```
1: input : The current population
2: Select parents  $P1$  and  $P2$  by the binary tournament selection scheme
3: for each event  $e_i$  of the child  $Ch$  do
4:   if  $rand(0, 1) < 0.5$  then
5:      $e_i$  of  $Ch \leftarrow$  the time slot allocated to  $e_i$  of  $P1$ 
6:   else
7:      $e_i$  of  $Ch \leftarrow$  the time slot allocated to  $e_i$  of  $P2$ 
8:   end if
9: end for
10: allocate rooms to all occupied time slots using the matching algorithm
11: output : A new child generated using  $Crossover()$ 
```

A uniform crossover operator is used with a probability $P_c = 0.8$. It first randomly assigns a time slot from one of the two parents to each event in the offspring and then allocates rooms to events in each non-empty time slot. The pseudo-code of the crossover operator.

Mutation

A mutation operator is used with a probability $P_m = 0.5$. It randomly selects a neighbourhood structure $N1$, $N2$, or $N3$, and makes a move in the selected neighbourhood to mutate an individual.

Evaluation

There are 50 runs for each run, the maximum run time t_{max} was set to 90 seconds for small instances, 900 seconds for medium instances, and 9000 seconds for the large instance

UCTP Instance	Best
S1	0
S2	4
S3	11
S4	21
S5	13
M1	17
M2	21
M3	19
M4	25
M5	23
lrc-2007	16002305

Table 3: Memetic Evaluation

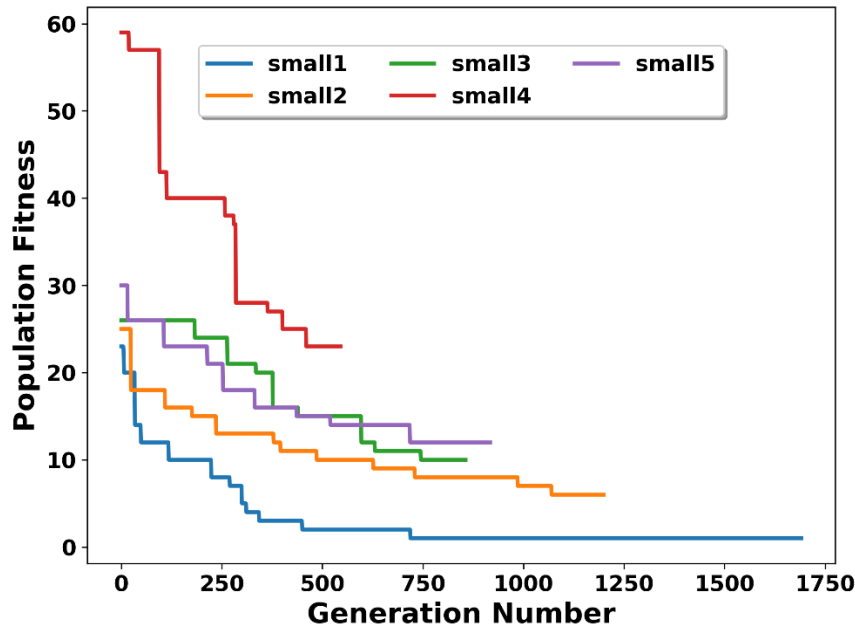


Figure 1: Memetic Convergence Chart

Algorithmic Enhancements and Architectural Optimizations

While the foundational literature establishes the theoretical basis for timetabling metaheuristics, the execution speed and solution quality of legacy solvers were inherently constrained by the software paradigms and hardware limitations of their era. The engine developed in this thesis achieves significant performance improvements over historical benchmarks through a combination of modern data-oriented architecture and advanced algorithmic escapement strategies. These optimizations can be categorized into four primary domains:

1. **Data-Oriented Architecture and Memory Locality:** Legacy solvers were frequently implemented using object-oriented paradigms in languages that rely heavily on dynamic memory allocation and garbage collection. In contrast, the optimization engine in this research utilizes a strict zero-allocation architecture during its core execution loops. By modeling the timetable using contiguous memory blocks (specifically lightweight `int16_t` arrays) and eliminating dynamic heap allocations, the system maximizes CPU cache hit rates. This ensures that processing cycles are exclusively dedicated to algorithmic optimization rather than memory management overhead.
2. **Constant-Time Delta Evaluation:** A significant computational bottleneck in standard timetabling algorithms is the necessity to execute a full state evaluation often requiring computationally expensive $O(E^2)$ operations or complete Bipartite Matching algorithms after every scheduled mutation. To bypass this, the implemented engine features a highly optimized delta evaluation mechanism (`get_scv_delta`). Instead of recalculating the entire timetable, this function isolates and computes the localized impact of a move in constant time. This temporal bypass allows the algorithm to skip heavy bipartite mathematics for the

vast majority of neighborhood explorations, exponentially increasing the number of candidate solutions evaluated per second.

- 3. Adaptive Search and Local Optima Escapement:** Standard Memetic Algorithms are prone to premature convergence, frequently becoming trapped when navigating flat areas of the fitness landscape. To overcome this, the engine employs an adaptive local search strategy. First, it permits the acceptance of "neutral moves" (sideways transitions that neither improve nor degrade the current fitness), allowing the algorithm to actively traverse plateaus rather than stagnating. Second, as the objective function approaches zero, the engine dynamically escalates its disruption mechanism utilizing an N_3 Kempe Chain neighborhood operator. This organic escalation effectively breaks the deadlocks that typically trap rigid, legacy solvers.

Chapter 4: Culture Algorithm

Introduction

Cultural Algorithms (CAs) are evolutionary optimization techniques introduced by Robert Reynolds to enhance the performance of traditional evolutionary algorithms. While conventional evolutionary algorithms rely mainly on genetic inheritance and natural selection to evolve solutions, Cultural Algorithms incorporate an additional learning mechanism inspired by theories of cultural evolution in sociology and archaeology.

The fundamental idea behind Cultural Algorithms is that evolution occurs at two levels. The first level, known as micro-evolution, represents the genetic inheritance process in which individuals evolve through operators such as selection, crossover, and mutation. The second level, known as macro-evolution, represents the accumulation and transmission of knowledge acquired by individuals across generations. This knowledge is preserved and shared within the population, allowing future generations to benefit from past experiences.

To model this dual inheritance process, Cultural Algorithms operate through two interconnected spaces: the Population Space and the Belief Space. The Population Space contains the candidate solutions that evolve over time, while the Belief Space stores useful knowledge extracted from high-quality individuals. Communication between these two spaces is achieved through the Accept Function, which updates the Belief Space using information from selected individuals, and the Influence Function, which uses stored knowledge to guide the search process.

By combining evolutionary search with knowledge-based guidance, Cultural Algorithms can improve convergence speed, enhance learning capabilities, and increase the likelihood of finding high-quality solutions. These advantages have made Cultural Algorithms an effective approach for solving complex optimization problems, including timetabling, scheduling, engineering design, and other combinatorial optimization tasks.

Proposed Approach

The approach proposed in this paper uses, in its population space, a population based on the evolutionary algorithm originally proposed. A pseudo-code with the main steps of the proposed cultural algorithm is shown in Algorithm 1. In our algorithm, we have considered three types of knowledge: situational, normative and domain knowledge. Also, we are using five variation operators: two of them use cultural knowledge (cultural mutation and repair) while the other three are designed to add the exploration component of the algorithm (interchange, sequencing and simple mutation). It is worth mentioning that only one of the exploration operators is applied to everyone.

Representation: The representation adopted to encode the solutions plays a very important role when applying an evolutionary computation technique. In this case, a matrix representation was adopted, where columns represent slots of time, and rows represent rooms for the events. This encoding was chosen because it can represent any feasible timetable and is easier to analyze the violation of some hard constraints, considering only one column at a time. **Exploration Operators:** The exploration operators are those that allow to maintain diversity of the population. They are listed next. The sequencing operator is similar to the one in [5], and its intention is to generate a large change in the individual since it interchanges two timeslots (this operator is the most destructive one used here). The interchange operator of [2], interchanges two events, and its purpose is to modify the individuals when the problems have in their feasible solutions the same number of places available and events to assign. The simple mutation operator changes the place of an event, and it is useful when the problems have more places available in their feasible solutions (without considering the last periods of the day) than events to assign. The last two operators make use of the matching algorithm to increase their rate of success.

	Pseudo-code of the cultural algorithm adopted
	Generate s random schedules (initial population)
	Compute the fitness of each individual in the initial population
	Initialize the belief space (copying the best individual to the situational belief space and create the normative matrix)
	repeat
	for each individual in current population do
	Apply cultural mutation operator
	switch (operator)
	case Interchange: Apply Interchange Operator
	case Sequencing: Apply Sequencing Operator
	case SimpleMutation: Apply Simple Mutation Operator
	end switch
	Apply repair operator (with domain knowledge)
	end for
	Selection process
	Update the belief space (with the individuals accepted)
	until the end condition is satisfied

The parameter control is a process, concurrent to the search of solutions, that allows values of the parameters to change during this process. We use a mechanism of parameter control in order to select the exploration operator (interchange, sequencing or simple mutation) to apply during the mutation process, using a roulette wheel and based on the success rate of each operator. This mechanism consists of updating the probability of each operator to be applied, following some simple rules. If the application of the operator number i results on an improvement of the fitness of the generated individual (with respect to his parent) ($f_{cur} < f_{prev}$) the update of the probabilities is made as follows: $operator[i] = operator[i] + \Delta variation$ where $operator[]$ is the array that contains the probabilities of the operators to be applied, $\Delta variation = \frac{f_{prev} - f_{cur}}{f_{prev} + f_{cur}}$ and $\forall j \in \{1, \dots, NumOper\}$ and $i \neq j$, $operator[j] = operator[j] - \frac{\Delta variation}{NumOper - 1}$ with $NumOper = 3$ in this case, because we have three operators.

When an operator i is applied and the present solution gets worse ($f_{cur} > f_{prev}$) ($f_{cur} < f_{prev}$) $operator[i] = operator[i] - \Delta variation * \alpha$ where $\alpha = \frac{Present\ time}{Total\ time}$ and $\forall j \in \{1, \dots, NumOper\}$ and $i \neq j$, $operator[j] = operator[j] + \frac{\Delta variation * \alpha}{NumOper - 1}$. The goal of incorporating the α factor is to maintain controlled the level of decrement, with the objective of not disturbing those operators whose decreasing ranks are much greater, like the sequencing operator. Initially, the 3 operators in competition start with the same probability of being chosen: $\forall i \in \{1, \dots, NumOper\}$, $operator[i] = 1/CantOper$. In order to assure that all operators always have a probability $\neq 0$ of being chosen, all values in $operator[]$ remain between $MinP\ rob = 0.1$ and $MaxP\ rob = 0.8$.

Mutation Operators with Cultural Influence: The operator begins selecting an event E and a position (r, t) to move it. This is done through different types of cultural influence. Situational Influence: With the situational influence each individual tries to follow a leader. Such a leader is the best individual found, and is stored in the situational belief space. The key idea is that the individual to be mutated becomes more similar to the leader after the mutation process. The mutation operator randomly selects an event E from the leader, and tries to inherit its position (r, t) to the individual. The situational belief space is updated at each generation. If the best individual of the current generation is better than the leader in the situational belief space, then the leader is replaced by that individual. Normative Influence: This type of influence is more complex. At each generation, the above average individuals are selected. The idea is to influence the individual to be mutated to inherit some of their characteristics. Before describing the procedure, we need the following definitions:

Definition 41 We define a ranking of events as the set of all the events ordered by the number of events with shared students among them. Thus, the event most connected with other events is the first in the ranking.

Definition 42 Given a population $P(g)$ of the generation g and the set S_g composed by the best s individuals of the generation g , we define M , where each element M_{ij} is the timeslot assigned to the event i in the individual j which belongs to S_g .

The operator proceeds as follows. The room r is fixed. The event is chosen from the ranking of events using a roulette wheel procedure which is biased to the most interconnected events. The new timeslot in the same room r is randomly selected from the matrix M , thus the most common timeslot t of the event E in M has the biggest probability of being selected. The hardest event to be assigned, from the constraints point of view, is the event that shares students with the largest number of events. The matrix M is updated at each generation g , after the selection of the set S_g (the above average individuals). Once an event E and the position (r, t) have been selected (by any of the cultural influences mentioned), the process of mutation continues as shown in Algorithm 2. First of all, the operator identifies the current position (r_E, t_E) of the event E in the individual to be mutated. If the new position selected (r, t) is empty and if it is feasible to place E there (from the hard constraints point of view), the current position of event E is modified to (r, t) . In case another event E_m is in (r, t) , the operator makes swapping moves to change E_m to another position, in order to release (r, t) .

Domain Knowledge: Our algorithm makes a post-processing procedure which uses the domain knowledge to modify individuals. In the timetabling problem, it is known that the best solution does not include events in the last timeslots of each day, thus the purpose of the repair operator is to try to move the events located in the last timeslots to the earliest ones, always satisfying the hard constraints.

mutation($E, (r, t)$) procedure, which implements mutation after the influence of cultural selection

```

1: mutation_finished = FALSE
2: identify the position  $(r_E, t_E)$  of the event  $E$  in the chromosome
3: while mutation_finished  $\neq$  TRUE or maxtries < 1000 do
4:   if the position  $(r, t)$  of the chromosome is empty then
5:     try to move the event  $E$  from  $(r_E, t_E)$  to  $(r, t)$ , satisfying the hard constraints
6:   else
7:     try a swapping move of the event  $E_m$  in  $(r, t)$ 
8:   end if
9:   if the position of  $E$  was changed then
10:    mutation_finished = TRUE
11:   end if
12: end while

```

Evaluation

To evaluate the performance of the proposed approach, the algorithm was run for 360 seconds on each benchmark instance. The best solutions obtained during the execution are summarized below.

Results on Small Instances

```
===== RESULT =====  
Hard violations: 0  
Soft violations: 34  
Fitness : 34  
Feasible : YES
```

Table 4: Cultural algorithm evaluation on small instances

Results on Medium Instances

```
===== RESULT =====  
Hard violations: 0  
Soft violations: 47  
Fitness : 47  
Feasible : YES
```

Table 5: Cultural algorithm evaluation on medium instances

Chapter 5: Multi-Objective UCTP

Introduction

Up until this point in the thesis, we have tackled the university course timetabling problem (UCTP) as a single-objective optimisation problem. As discussed in Chapter 2, for more than forty years, the UCTP has been studied as a single-objective optimisation problem and yet there is no formulated general solving technique for the problem due to the complex and highly-constrained nature of the problem. It is very difficult to find a general and effective solution for timetabling owing the diversity of the problem, the variance of constraints, and special objectives from university to university according to their own particular characteristics.

In this part of our research, we will now switch our effort over to solve the problem as a multi-objective optimisation problem (MOOP) in the hope of dealing with multiple and conflicting objectives simultaneously for the UCTP. In this chapter, we first develop a framework of multi-objective evolutionary algorithms (MOEAs) to solve the multi-objective university course timetabling problem (MOUCTP).

Matching Algorithm

We treat this as a classic Maximum Bipartite Matching problem. We have two disjoint sets: Events (E) and Rooms (R). Edges only exist between an Event and a Room if the room is suitable (e.g., right size, right equipment). The network flow algorithm finds the largest possible number of valid pairs.

The algorithmic mandate is to assign each overlapping event in E_{t_j} to a distinct room in R such that the capacity and feature requirements are unconditionally satisfied. This assignment puzzle maps perfectly onto the classical Maximum Cardinality Bipartite Matching problem in graph theory. The algorithm must discover a matching configuration that successfully places the absolute maximum number of events into valid rooms. Rossi-Doria et al. explicitly document that this resolution was achieved utilizing a "deterministic network flow algorithm".

Graph-Theoretic Construction

To initiate the matching for a specific timeslot t_j , an unweighted bipartite graph $G = (U, V, E)$ is constructed dynamically in memory. The components of the graph are established as follows:

- Partition U : Represents the set of events scheduled in the current timeslot ($U = E_{t_j}$)
- Partition V : Represents the set of all available rooms ($V = R$)
- Edges E : An unweighted edge (u, v) is instantiated between event node u and room node v if and only if room v fully satisfies the physical requirements of event u . Specifically, the seating capacity of room v must be strictly greater than or equal to the total number of students enrolled in event u , and the feature set of room v must be a complete superset of the feature set required by event u .

Transformation to Network Flow

The identification of a maximum cardinality matching within a bipartite graph can be efficiently resolved by transforming the structure into a maximum network flow problem, which leverages the Ford-Fulkerson theorem. A directed flow network $G' = (U \cup V \cup s, t, E')$ is generated by introducing two artificial nodes: a super-source node s and a super-sink node t .

The edge configuration is subsequently modified:

- Directed edges with an absolute capacity of 1 are established from the super-source s to every event node $u \in U$.

- Directed edges with an absolute capacity of 1 are established from every room node $v \in V$ to the super-sink t .
- Every original undirected valid connection edge $(u, v) \in E$ is converted into a directed edge flowing from u to v also constrained with a capacity of 1.
- According to the classical Max-Flow Min-Cut theorem, the integral maximum flow of units pushed from s to t through the directed network G' corresponds exactly to the maximum cardinality matching in the original bipartite graph G . The deterministic network flow algorithm iterates, pushing flow through augmenting paths within this network until no further unutilized paths can be discovered.

The Critical Deterministic Fallback Mechanism

While the maximum cardinality matching guarantees the optimal assignment of events to valid rooms, instances arise where $|M| < |E_{tj}|$. This scenario occurs frequently in heavily constrained data instances, or during the early, highly stochastic exploratory phases of a metaheuristic where the temporal timeslot assignment might be disastrously suboptimal. In these instances, events are left unplaced because there are simply no remaining valid rooms, or because the metaheuristic has scheduled far too many events in a single timeslot.

If an algorithm leaves events unassigned, the timetable is incomplete, making objective fitness evaluation impossible. To resolve this, Rossi-Doria et al. implemented a strict, uncompromising deterministic fallback protocol to handle overflow events. This ensured that a fully defined timetable, albeit a highly illegal one containing severe hard constraint violations, was always produced for evaluation.

The fallback procedure is explicitly documented in the literature:

"If there are still unplaced events left, it takes them in label order and puts each one into the room time slots and rooms to all the events."

This tie-breaking rule was paramount to the success of the comparative study. Because trajectory-based metaheuristics like Tabu Search and Simulated Annealing rely on deterministic fitness evaluations to map the contours of the search space, the matching algorithm could not rely on stochastic or randomized assignment for overflow events. By systematically forcing unplaced events into valid rooms in label order, the algorithm artificially violated the "one event per room" constraint, thereby generating a measurable, predictable penalty. This continuity maintained a smooth fitness gradient that the metaheuristics could effectively optimize against.

Local Search Strategies

Local Search 2

```
1: input : Individual I after LS1 is applied
2:  $S :=$  randomly select a  $tp$  percentage of time slots from the total time slots of  $T$ 
3: for each time slot  $t_i \in S$  do
4:   for each event  $j$  in time slot  $t_i$  do
5:     calculate the penalty value of event  $j$ 
6:   end for
7:   sum the total penalty value of events in time slot  $t_i$ 
8: end for
9: select the time slot  $w_t$  with the biggest penalty value from  $S$ 
10: for each event  $i$  in  $w_t$  do
11:   calculate a move of event  $i$  in the neighbourhood structure  $N1$ 
12:   apply the matching algorithm to the time slots affected by the move
13:   compute the penalty of event  $i$  and delta-evaluate the result
14: end for
15: if all the moves together reduce hard or soft constraint violations then
16:   apply the moves
17: else
18:   delete the moves
19: end if
20: output : A possibly improved individual I
```

LS2 first randomly selects a tp percentage of time slots from the total set of time slots T . Rather than selecting the worst time slot from all time slots, LS2 randomly selects a subset and then picks the worst one within that subset. This reduces computation time, because penalty values must be calculated for examined time slots.

Then LS2 computes the penalty of each selected time slot and chooses the worst time slot w_t (the one with the largest penalty) for LS operations. In this way, LS2 aims to improve the result produced by LS1.

After selecting w_t , LS2 tries an $N1$ move for each event in w_t , and checks each event's penalty value before and after the move. If all moves in w_t together reduce hard and/or soft constraint violations, LS2 applies all moves; otherwise, it discards them.

This allows LS2 not only to examine the worst time slot but also to reduce penalties for some events by moving them to other time slots. In general, LS2 can enhance individuals in the population and improve feasible timetable quality by reducing constraint violations.

Local Search 5

	Local Search Strategy 5 (LS5)
1:	input : Individual I selected from the population
2:	while Termination condition not reached do
3:	for $i = 1$ to the total number of events do
4:	if event i is infeasible then
5:	if there is untried move left then
6:	calculate the next move (first in $N1$, then in $N2$, then in $N3$, and finally in $N7$)
7:	apply the matching algorithm to the time slots affected by the move and delta-evaluate the result.
8:	if the move reduces hard constraint violation then
9:	make the move
10:	end if
11:	end if
12:	end if
13:	end for
14:	if any hard constraint violations remain then
15:	terminate LS3
16:	else
17:	for $i = 1$ to total number of events do
18:	if event i has soft constraint violation then
19:	if there is untried move left then
20:	calculate the next move (first in $N1$, then in $N2$, then in $N3$, and finally in $N7$)
21:	apply the matching algorithm to the time slots affected by the move and delta-evaluate the result
22:	if the move reduces soft constraints violation then
23:	make the move
24:	end if
25:	end if
26:	end if
27:	end for
28:	end if
29:	end while
30:	output : A possibly improved individual I

Local Search Strategy 5 (LS5) is an improvement procedure applied to a timetable solution after it is generated, either during initialization or after mutation. The objective of LS5 is to improve the quality of the solution by first eliminating hard constraint violations and then reducing soft constraint violations.

LS5 uses four neighborhood structures:

- **N1:** Moves a single event from its current time slot to a different time slot.
- **N2:** Swaps the time slots of two events.
- **N3:** Permutes three events assigned to three different time slots.
- **N7:** Swaps the time slots of two consecutive events with another pair of consecutive events.

The main advantage of LS5 over LS1 is the addition of **N7**, which allows the algorithm to perform larger modifications to the timetable and explore the search space more effectively.

Phase 1: Hard Constraint Improvement

In the first phase, LS5 focuses exclusively on hard constraints, which represent mandatory scheduling requirements. The algorithm examines each event in the timetable and attempts to remove any hard constraint violations.

For each event, LS5 explores the neighborhood structures sequentially:

1. **N1**: Try moving the event to different time slots.
2. **N2**: If no improvement is found, try swapping the event with other events.
3. **N3**: If necessary, apply three-event permutations.
4. **N7**: Finally, swap pairs of consecutive events with other consecutive pairs.

After every move:

- The matching algorithm is applied to reassign rooms for the affected events.
- Delta evaluation is performed to calculate the change in hard and soft constraint violations.
- The move is accepted only if it improves the solution by reducing hard constraint violations.

This process continues until an improvement is found or a predefined maximum number of search steps is reached.

If hard constraint violations still remain after all possible moves have been examined, LS5 terminates. Otherwise, it proceeds to the second phase.

Phase 2: Soft Constraint Improvement

Once a feasible timetable has been obtained, meaning that all hard constraints are satisfied, LS5 begins optimizing soft constraints.

The same neighborhood structures (**N1**, **N2**, **N3**, and **N7**) are explored again for each event. However, in this phase:

- Hard constraints must remain satisfied.
- Moves are accepted only if they reduce soft constraint violations.

As in the first phase, every move is followed by room reallocation using the matching algorithm and delta evaluation to measure its impact on the solution.

Output

At the end of the search process, LS5 produces a timetable that:

- Satisfies all hard constraints whenever possible.
- Has fewer soft constraint violations.
- Is generally of higher quality than the original solution.

In summary, LS5 is a two-stage local search algorithm that improves timetable feasibility and quality using four neighborhood structures (N1, N2, N3, and N7), with N7 providing additional exploration capabilities through larger timetable modifications.

The GS Strategy

Data Structures MEM_i (i = 1, 2, 3)

	<i>ConstructMEM3(P)</i> – Constructing data structures
1:	input: The input population P of size N
2:	$Q \leftarrow$ select the best $\alpha \times N$ individuals in P
3:	for each individual I_j in Q do
4:	for each objective i do
5:	if $f_i(I_j) = 0$ then
6:	for each event e_k in I_j do
7:	calculate the penalty value of event e_k from I_j
8:	if e_k is feasible (i.e., e_k has zero constraint violation) then
9:	add the pair of room and time slot (r_{e_k}, t_{e_k}) assigned to e_k into the list l_{e_k} in MEM_i
10:	end if
11:	end for
12:	end if
13:	end for
14:	end for
15:	output: Updated data structures MEM_i ($i = 1, 2, 3$)

Guided Search (GS) Strategy – MEM Data Structures

The Guided Search (GS) strategy is designed to help the evolutionary algorithm generate better offspring by learning from high-quality solutions found during the search process.

To achieve this, the algorithm creates three memory structures called:

- **MEM1**
- **MEM2**
- **MEM3**

Each memory is associated with one of the three optimization objectives of the problem.

The purpose of these memories is to store useful scheduling information extracted from the best individuals in the population. This stored knowledge is later used to guide the generation of new solutions, improving both convergence and solution quality.

Structure of MEMi

Each memory structure contains:

- A list of events.
- For every event, a list of feasible **(Room, Time Slot)** pairs.

Therefore, MEMi acts as a repository of good event assignments that performed well with respect to objective i.

Construction of MEM Structures

The memories are not built only once; they are periodically reconstructed every τ **generations** to ensure that they contain the most up-to-date information discovered by the search process.

The construction process follows these steps:

Step 1: Select the Best Individuals

From the current population **P**, the algorithm selects:

$$\alpha \times N$$

best individuals based on their dominance ranking, where:

- **N** = population size.
- α = selection ratio.

These selected individuals form a subset called **Q**.

Step 2: Examine Objective Values

For every individual I_j in **Q**:

- The algorithm checks its objective values.
 - If an objective $f_i(I_j)$ equals **0**, it means that this individual has achieved the best possible value for objective i.
-

Step 3: Check Individual Events

For such individuals:

- Every event is examined separately.
- The penalty value of each event is calculated.

If an event has:

$$Penalty = 0$$

it means that the event does not contribute to any violation related to that objective.

Step 4: Store Good Event Assignments

For every zero-penalty event:

- Its assigned **room** and **time slot** information is stored in the corresponding memory structure **MEM_i**.

For example:

- Events that contribute perfectly to Objective 1 are stored in **MEM1**.
 - Events that contribute perfectly to Objective 2 are stored in **MEM2**.
 - Events that contribute perfectly to Objective 3 are stored in **MEM3**.
-

Why is MEM Important?

Instead of remembering only complete good solutions, the algorithm remembers the **best parts of solutions**.

This provides two major benefits:

1. **Elitism**
 - Preserves high-quality scheduling decisions discovered during the search.
2. **Guided Search**
 - Helps future offspring inherit good event assignments.
 - Directs the search toward promising regions of the solution space.
 - Increases the probability of generating high-quality timetables.

Generating a Child Using the Guided Search (GS) Strategy

The Guided Search (GS) strategy is an alternative method for generating offspring in the evolutionary algorithm. Instead of relying solely on crossover, GS uses the

knowledge stored in the memory structures (**MEM1, MEM2, and MEM3**) to construct new solutions.

When a new child needs to be generated, the algorithm first produces a random number ρ between 0 and 1.

- If $\rho < \gamma$, the child is generated using the **Guided Search (GS)** strategy.
- Otherwise, a standard **crossover** operation is performed.

Here, γ controls the probability of using Guided Search instead of crossover.

	<i>GuidedSearch3()</i> – Generating a child by GS
1:	input: The data structures $MEM_i (i = 1, 2, 3)$
2:	randomly select one data structure MEM_j
3:	$E_s :=$ randomly select $\beta * n$ events
4:	for each event e_k in E_s do
5:	randomly select a pair of room and time slot from the list l_{e_k} in MEM_j
6:	assign the selected pair to event e_k for the child
7:	end for
8:	for each remaining event e_k not in E_s do
9:	assign a random time slot and room to event e_k
10:	end for
11:	output: A child generated using $MEM_i (i = 1, 2, 3)$

Child Generation Process

When the GS strategy is selected, the algorithm performs the following steps:

Step 1: Select a Memory Structure

One of the three memory structures (**MEM1, MEM2, or MEM3**) is randomly selected.

This selected memory acts as the knowledge source for generating part of the child solution.

Step 2: Select a Set of Events

A subset E_s containing:

$$\beta \times n$$

events is randomly selected, where:

- n = total number of events.
- β = percentage of events to be generated from memory.

These events will inherit information from the selected memory structure.

Step 3: Generate Events from Memory

For each event e_k in E_s :

- The algorithm checks the corresponding list of feasible **(Room, Time Slot)** pairs stored in the selected memory.
- One pair is chosen randomly.
- The selected room and time slot are assigned to the event in the child solution.

This allows the child to inherit high-quality scheduling decisions discovered during previous generations.

Step 4: Handle Missing Information

If an event does not have any stored information in the selected memory structure:

- A room and time slot are assigned randomly from the available options.

This ensures that every selected event receives a valid assignment.

Step 5: Complete the Child

After assigning all events in E_s :

- The remaining events that were not selected are assigned random feasible rooms and time slots.

Once all events have been assigned, a complete child timetable is obtained.

Genetic Operators

The proposed framework employs a set of genetic operators to evolve the population and generate high-quality timetable solutions. These operators include constraint handling, selection, crossover, and mutation mechanisms.

Objective Functions and Constraints Handling

The objective of the Multi-Objective University Course Timetabling Problem (MOUCTP) is to minimize three categories of soft constraint violations while satisfying all hard constraints.

Since the MOUCTP is a highly constrained optimization problem, the framework adopts the constraint-handling mechanism of the NSGA-II algorithm. This mechanism is based on the concepts of **feasibility** and **Pareto dominance**.

When comparing two solutions, a solution a is considered better than solution b (i.e., constrained-dominates b) if one of the following conditions holds:

1. **Solution a is feasible while solution b is infeasible.**
2. **Both solutions are feasible, and solution a Pareto-dominates solution b .**
3. **Both solutions are infeasible, but solution a has fewer constraint violations and dominates solution b .**

This strategy ensures that feasible solutions are always preferred over infeasible ones, while still allowing the search process to improve infeasible solutions until feasibility is achieved.

Selection Mechanism

The selection mechanism depends on the Multi-Objective Evolutionary Algorithm (MOEA) used within the framework. A commonly adopted approach is **Binary Tournament Selection without Replacement**.

In this method:

- Two individuals are randomly selected from the population.
- Their ranks and crowding distances are compared.
- The better individual is selected as a parent.
- The selected individual is temporarily removed from the candidate pool to avoid repeated selection during the same tournament round.

This mechanism balances solution quality and population diversity, which are essential for effective multi-objective optimization.

Crossover Operator

After selecting two parent solutions, crossover is applied with a probability:

$$P_c = 1$$

which means crossover is always performed.

The framework uses a **Uniform Crossover Operator** to generate offspring.

The process is as follows:

1. For each event in the timetable, the offspring randomly inherits the time slot assignment from one of the two parents.

2. After all time slots are assigned, room assignments are determined using the matching algorithm.
3. The matching algorithm allocates suitable rooms to all events in each occupied time slot while minimizing room allocation conflicts.

The crossover operator combines useful characteristics from both parents and promotes the creation of diverse offspring solutions.

Mutation Operator

After crossover (or Guided Search generation), the newly created offspring undergoes mutation with probability:

$$P_m$$

The mutation operator randomly selects one of the available neighborhood structures:

- **N1:** Move an event to a different time slot.
- **N2:** Swap the time slots of two events.
- **N3:** Apply a permutation among three events.
- **N7:** Swap two consecutive events with another pair of consecutive events.

A randomly chosen move is then applied to the offspring.

The purpose of mutation is to:

- Introduce diversity into the population.
- Prevent premature convergence.
- Explore new regions of the search space.
- Improve the chances of discovering higher-quality timetable solutions.

Guided Search NSGA-II (GSNSGA)

GSNSGA is a hybrid multi-objective evolutionary algorithm developed by integrating the Guided Search (GS) strategy with the NSGA-II framework. The algorithm combines the exploration and diversity preservation capabilities of NSGA-II with a memory-based search mechanism that utilizes knowledge extracted from high-quality solutions found during the evolutionary process.

The algorithm begins by generating an initial random population P_g of size N . Each individual in the population represents a complete timetable solution. After the population is created, Local Search strategies (LS1 and LS5) are applied to each individual to improve its quality by reducing hard and soft constraint violations.

Non-Dominated Sorting and Crowding Distance

After evaluating all individuals, GSNSGA applies the non-dominated sorting procedure of NSGA-II.

For each solution P_i , two values are calculated:

- **Domination Count (n_{P_i})**: the number of solutions that dominate P_i .
- **Dominated Set (S_{P_i})**: the set of solutions dominated by P_i .

Using these values, individuals are classified into non-dominated fronts:

- Solutions with domination count equal to zero belong to the **first front** and receive **Rank 1**.
- After assigning Rank 1, the domination counts of solutions dominated by these individuals are reduced.
- Any solution whose domination count becomes zero is assigned to the **second front** and receives **Rank 2**.
- This process continues until all individuals are assigned to a front.

Once ranking is completed, the **crowding distance** is calculated for each individual within the same front. The crowding distance estimates the density of surrounding solutions and helps preserve diversity by favoring individuals located in less crowded regions of the objective space.

Construction of Memory Structures

A key feature of GSNSGA is the use of three memory structures:

- MEM1
- MEM2
- MEM3

Each memory corresponds to one of the three optimization objectives.

Every τ generations, including the initial generation, these memory structures are constructed or reconstructed using the best individuals in the current population. The memories store high-quality event assignments that can later be used to guide offspring generation.

This mechanism allows the algorithm to preserve useful knowledge discovered during the search and exploit it in future generations.

Child Generation

At each generation, a child population Q_g is created.

For every offspring, a random number is generated and compared with the probability parameter γ :

- If the random value is less than γ , the **Guided Search (GS)** strategy is used.
- Otherwise, the standard **crossover** operator is applied.

Guided Search Generation

When Guided Search is selected:

1. One memory structure (MEM1, MEM2, or MEM3) is chosen randomly.
2. A subset of events is selected.
3. These events inherit room–time slot assignments from the selected memory.
4. Remaining events are assigned randomly.

This process allows offspring to inherit high-quality building blocks from previously successful solutions.

Crossover Generation

When crossover is selected:

1. Two parents are chosen using **crowded tournament selection**.
2. In each tournament, two individuals are randomly selected.
3. The winner is determined based on:
 - Lower rank value.
 - Higher crowding distance when ranks are equal.
4. The selected parents undergo uniform crossover to produce an offspring.

Mutation and Local Improvement

After a child is generated, a mutation operation is applied with probability P_m .

The mutation randomly selects one of the neighborhood structures:

- N1
- N2
- N3

- N7

and performs the corresponding move to introduce diversity into the population.

Population Update

After all offspring are generated:

1. The parent population P_g and child population Q_g are merged to form R_g .
2. The merged population contains $2N$ individuals.
3. Non-dominated sorting and crowding distance calculations are performed again.
4. The best N individuals are selected from R_g according to:
 - Rank (lower is better)
 - Crowding Distance (higher is better)

The selected individuals form the next population P_{g+1} .

This elitist mechanism ensures that high-quality solutions are never lost during evolution.

Termination

The evolutionary process continues until a predefined termination condition is reached, such as:

- Maximum execution time.
- Maximum number of generations.
- Desired solution quality.

The final output of GSNSGA is a set of **Pareto-optimal (non-dominated) timetable solutions** representing different trade-offs among the three objectives.

	Guided Search NSGA-II (GSNSGA)
1:	input: A problem instance I
2:	set the generation counter $g := 0$
3:	initialise a population P_g of N solutions
4:	<i>apply LS1 and LS2 to individuals in P_g</i>
5:	evaluate the individuals in P_g
6:	assign rank and crowding distance for individuals in P_g
7:	while the termination condition is not reached do
8:	if $(g \bmod \tau) == 0$ then
9:	<i>apply ConstructMEM3(P_g) to construct MEM_i ($i = 1, 2, 3$)</i>
10:	end if
11:	create a child population Q : use <i>GuidedSearch3()</i> or crossover with a probability γ to generate each child independently, followed by mutation with a probability P_m
12:	<i>apply LS1 and LS2 to individuals in Q_g</i>
13:	evaluate the child solutions in Q_g
14:	merge the child and parent populations into a combined population $R_g := P_g \cup Q_g$
15:	assign rank and crowding distance for individuals in R_g
16:	create a new population from R_g based on rank and crowding distance
17:	$g := g + 1$
18:	end while
19:	output: Non-dominated set of solutions

Evaluation

For a thesis/report, you should **not include all 20 runs**. Instead, present a concise summary table containing the main statistics (Best, Average, Worst, Standard Deviation, and Feasibility Rate).

Results on Small Instances

```
=====
SUMMARY STATISTICS (20 runs)
=====

Feasibility:
Runs with feasible solutions: 20 / 20
Best SCV (feasible runs only): 5
Avg SCV (feasible runs only): 10.85
Worst SCV (feasible runs only): 20

All-run statistics (best solution per run):
-----
Metric      Best      Average    Worst      Std Dev
-----
Total SCV   5         10.85     20         4.38
f1 (S1)     0         0.00      0          0.00
f2 (S2)     0         0.55      1          0.51
f3 (S3)     4         10.30     19         4.50
-----

Best run: Run 8 (seed=49) | SCV=5 | f1=0 | f2=1 | f3=4 | FEASIBLE
```

Table 6: GSNASGA evaluation on small instances

Results on Medium Instances

=====				
SUMMARY STATISTICS (20 runs)				
=====				
Feasibility:				
Runs with feasible solutions: 18 / 20				
Best SCV (feasible runs only): 114				
Avg SCV (feasible runs only): 206.56				
Worst SCV (feasible runs only): 318				
All-run statistics (best solution per run):				

Metric	Best	Average	Worst	Std Dev

Total SCV	114	205.70	318	49.82
f1 (S1)	8	75.75	159	41.53
f2 (S2)	55	99.85	135	18.35
f3 (S3)	11	30.10	52	11.64

Best run: Run 8 (seed=49) SCV=114 f1=8 f2=81 f3=25 FEASIBLE				

Table 7: GSNASGA evaluation on medium instances

Chapter 6: Hybrid Genetic Algorithm with Tabu Search (HGATS)

Introduction

The Post-Enrolment Course Timetabling Problem (PECTP) is a highly constrained combinatorial optimization problem that requires assigning events to appropriate timeslots and rooms while satisfying a set of hard and soft constraints. Due to the large search space and the complexity of the constraints, obtaining high-quality timetables using a single optimization technique is often difficult.

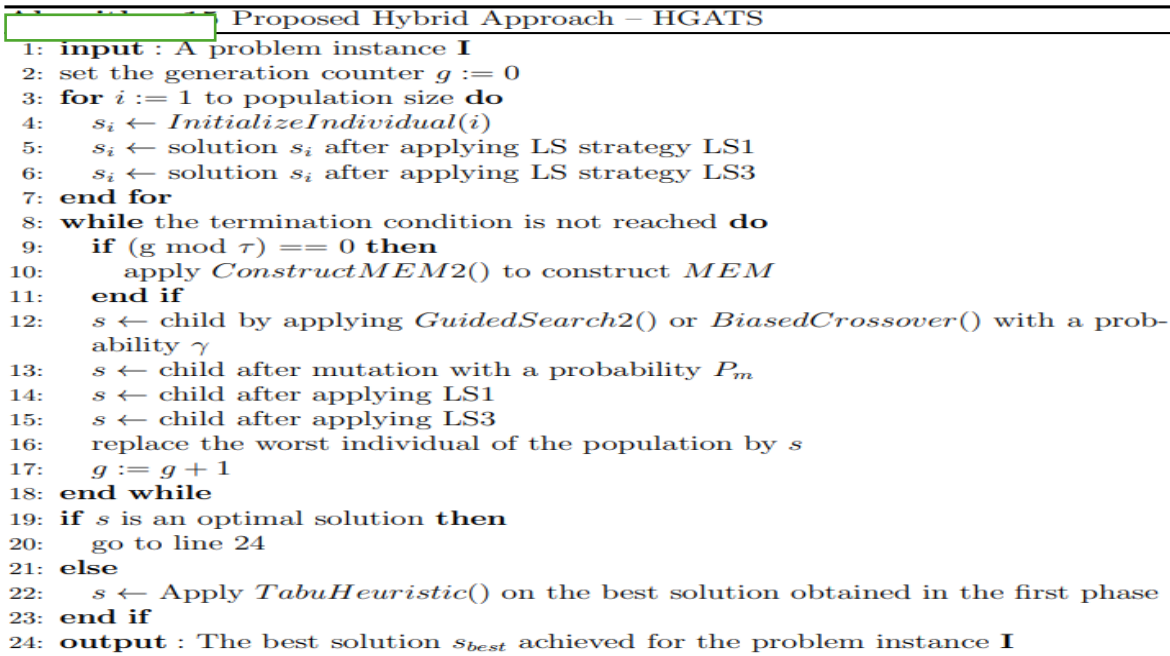
To address this challenge, a Hybrid Genetic Algorithm with Tabu Search (HGATS) is employed. The proposed approach combines the exploration capability of Genetic Algorithms with the exploitation capability of Tabu Search. The Genetic Algorithm explores diverse regions of the search space and generates promising candidate solutions, while Tabu Search intensifies the search around high-quality solutions to further improve timetable quality.

The HGATS framework consists of two consecutive phases. In the first phase, a Guided Search Genetic Algorithm (**GSGA**) generates a population of candidate timetables and improves them through genetic operators and local search mechanisms. In the second phase, the best solution obtained from GSGA is refined using **Tabu Search** to reduce constraint violations and improve the overall fitness value.

By integrating both population-based and trajectory-based optimization techniques, **HGATS** achieves a balance between diversification and intensification, enabling the generation of high-quality timetables for benchmark **PECTP** instances.

The HGATS Framework

The proposed **HGATS** framework combines a Guided Search Genetic Algorithm with a Tabu Search improvement phase. Figure X illustrates the overall workflow of the proposed approach



The algorithm begins by reading the benchmark instance and generating an initial population of feasible or near-feasible timetables. Each individual timetable is evaluated according to a fitness function that considers both hard and soft constraint violations. The population then evolves through guided search strategies, crossover, mutation, and local search operators.

After a predefined number of generations, the best individual produced by the GSGA phase is passed to the Tabu Search phase. Tabu Search explores neighboring solutions while maintaining a tabu list to avoid cycling and encourage exploration of new regions of the search space. The best timetable obtained during the search is returned as the final solution.

Phase I: Guided Search Genetic Algorithm (GSGA)

The first phase of HGATS is responsible for exploring the search space and generating promising candidate timetables. The algorithm maintains a population of individuals, where each individual represents a complete timetable solution.

The population evolves iteratively through evaluation, selection, crossover, mutation, and local search operations. Additionally, a guided search mechanism supported by memory structures is used to preserve useful information discovered during the search process.

Population Initialization

The initial population is generated by assigning events to available timeslot-room pairs while attempting to satisfy hard constraints whenever possible. Each generated timetable represents a complete solution and is stored as an individual in the population.

Population diversity is important during this phase because it allows the algorithm to explore multiple regions of the search space. Therefore, different random assignments are used to generate distinct individuals.

The generated population serves as the starting point for the evolutionary search process.

Fitness Evaluation

Each timetable is evaluated using a fitness function that considers both hard and soft constraint violations.

Hard constraints represent mandatory requirements that must be satisfied to produce a feasible timetable. Violating a hard constraint significantly increases the fitness penalty.

Soft constraints represent quality preferences that should be satisfied whenever possible. Although soft constraint violations do not make a timetable infeasible, they reduce timetable quality.

The fitness value is calculated as:

$$Fitness = (HardViolations \times 10^6) + SoftViolations$$

This formulation ensures that feasible solutions are always preferred over infeasible ones, regardless of their soft constraint penalties.

Guided Search and MEM Structure

To improve search efficiency, HGATS employs a guided search strategy supported by memory structures (MEM).

The memory structures store useful scheduling information extracted from high-quality solutions discovered during previous generations. This information guides the creation of new offspring by encouraging assignments that frequently appear in good timetables.

The guided search mechanism reduces randomness during evolution and increases the probability of generating high-quality offspring. As a result, the algorithm converges faster toward promising regions of the search space.

Crossover and Mutation Operators

Crossover Operator

The crossover operator combines scheduling information from two parent timetables to create new offspring. Events assigned in high-quality regions of both parents are preserved whenever possible.

This operation promotes information exchange between individuals and allows beneficial characteristics to propagate through the population.

Mutation Operator

The mutation operator randomly selects one of the available neighborhood structures and applies a move to perturb the offspring solution.

Mutation helps maintain population diversity and prevents premature convergence to local optima.

Together, crossover and mutation balance exploration and exploitation during the evolutionary process.

Local Search Improvement

Local Search Improvement

After the offspring solution is generated and subjected to mutation, two local search strategies, denoted as **LS3** and **LS4**, are applied to further improve its quality. The purpose of these local search procedures is to reduce hard and soft constraint violations and enhance the overall fitness of the timetable before it is introduced into the population.

LS3 operates on individual events and explores multiple neighborhood structures (**N1–N4**) to resolve conflicts and improve event assignments. The search process first focuses on eliminating hard constraint violations. Once a feasible solution is obtained, it attempts to reduce soft constraint penalties without introducing new conflicts.

LS4 provides an additional improvement mechanism by focusing on precedence-related constraints and problematic timeslots. It utilizes neighborhood structures **N5** and **N6** to relocate events, improve event ordering, and reduce penalties associated with heavily constrained regions of the timetable.

By combining **LS3** and **LS4**, the algorithm is able to intensify the search around promising solutions and quickly refine newly generated offspring. This hybrid local improvement process increases the likelihood of producing high-quality feasible timetables and accelerates the convergence of the overall HGATS framework.

Phase II: Tabu Search

Although the **GSGA** phase produces high-quality solutions, additional improvement can often be achieved through intensive local optimization. Therefore, the best timetable obtained from GSGA is passed to a Tabu Search procedure.

Tabu Search explores neighboring solutions while maintaining memory structures that prevent the search from repeatedly visiting the same regions of the search space.

This phase focuses on intensification and refinement of the best timetable discovered during the evolutionary search.

Neighborhood Generation

The neighborhood of a timetable consists of solutions obtained through small modifications to event assignments.

Typical neighborhood moves include:

- Moving an event to a different timeslot.
- Moving an event to a different room.
- Swapping two events.
- Reassigning conflicting events.

Each generated neighbor is evaluated using the fitness function.

The best admissible neighbor is selected as the next current solution.

Tabu List Management

To prevent cycling, Tabu Search maintains a tabu list containing recently performed moves.

Moves stored in the tabu list are temporarily prohibited for a predefined number of iterations. This prevents the algorithm from repeatedly revisiting previously explored solutions.

An aspiration criterion is also used. If a tabu move produces a solution better than the current best solution, the move is allowed despite its tabu status.

The tabu list therefore balances diversification and intensification during the search process.

Solution Improvement

At each iteration, the algorithm evaluates candidate neighbors and selects the most promising admissible move.

The current solution is updated continuously, while the best solution found throughout the search is preserved separately.

The search terminates when a predefined stopping criterion is reached, such as:

- Maximum number of iterations.
- Maximum execution time.
- No improvement for a specified number of iterations.

The best timetable obtained during this phase is returned as the final solution of the HGATS framework.

HGATS Algorithm

The overall workflow of HGATS can be summarized as follows:

Algorithm 6.1: HGATS Procedure

1. Read benchmark instance.
2. Generate initial population.
3. Evaluate population fitness.
4. Apply guided search mechanism.
5. Perform crossover and mutation.
6. Apply local search improvement.
7. Repeat until the generation limit is reached.
8. Select the best GSGA solution.
9. Initialize Tabu Search using the best solution.
10. Generate neighboring solutions.
11. Update tabu list.
12. Accept the best admissible move.
13. Repeat until the stopping criterion is satisfied.
14. Return the best timetable found.

Evaluation

The proposed **HGATS** algorithm was evaluated using benchmark instances from the International Timetabling Competition 2007 (**ITC-2007**) Post-Enrolment Course Timetabling dataset.

The benchmark set consists of 24 problem instances divided into three categories:

Early Instances

dataset1, dataset2, dataset3, dataset4, dataset5, dataset6, dataset7, and dataset8.

Late Instances

dataset9, dataset10, dataset11, dataset12, dataset13, dataset14, dataset15, and dataset16.

Hidden Instances

dataset17, dataset18, dataset19, dataset20, dataset21, dataset22, dataset23, and dataset24.

For experimental evaluation, the first ten benchmark instances were selected. Each instance was executed **20** independent times in order to reduce the influence of stochastic behavior and provide statistically meaningful results.

The maximum runtime for each run was fixed at **600** seconds, following the time limit used in the **ITC-2007** competition. Therefore, each benchmark instance received a total computational budget of **12,000** seconds.

The performance of **HGATS** was evaluated using the following measures:

- Best Distance to Feasibility (Best DF)
- Average Distance to Feasibility (Avg DF)
- Best Soft Constraint Penalty (Best SCP)
- Average Soft Constraint Penalty (Avg SCP)
- Average Execution Time
- Best Fitness Value
- Average Fitness Value

Results

Table 8: HGATS Results

Instance	Best DF	Avg DF	Best SCP	Avg SCP	Best Fitness
comp-2007-2-1	0	2	2563	2650.9	2563
comp-2007-2-2	1	6.1	0	0	1000000
comp-2007-2-3	0	0	1209	1542.75	1209

comp-2007-2-4	0	0	1653	1944.15	1653
comp-2007-2-5	0	0	1105	1315.7	1105
comp-2007-2-6	0	0	1026	1200.8	1026
comp-2007-2-7	0	0	523	737	523
comp-2007-2-8	0	0	905	980.95	905
comp-2007-2-9	1	5.7	0	0	1000000
comp-2007-2-10	4	10.65	0	0	4000000

Chapter Summary

This chapter presented HGATS, a hybrid optimization approach that combines a Guided Search Genetic Algorithm with Tabu Search for solving the Post-Enrolment Course Timetabling Problem. The first phase employs GSGA with guided search, memory structures, crossover, mutation, and local search strategies to explore promising regions of the search space. The second phase applies Tabu Search to further refine the best timetable produced by the genetic search process.

The proposed approach was evaluated using benchmark instances from the **ITC-2007** dataset. Multiple independent runs were conducted under a fixed execution time limit to assess the robustness and effectiveness of the algorithm. Experimental results demonstrate that the combination of guided evolutionary search and tabu-based intensification provides a powerful framework for producing high-quality timetables while effectively handling the hard and soft constraints of the **PECTP**.

Chapter 7: System Architecture

Functional Requirements

Admin:

- Login:

Admin can login to the system and interact with our features by typing the admin email and the password.

- Manage dashboard:

Admin can see the total number of features, the active rooms, the total number of instructors, total number of courses and total number of students.

- Manage features:

Admin can view, add and view actions on every feature.

- Manage rooms:

Admin can add rooms, view the rooms with their ids, the capacity of each of them, the features of each room and view actions on every room.

- Manage courses:

Admin can add courses, view the courses with their ids, the instructors of each of them, the number of students registered for each course and view actions on every course.

- Generate timetable:

Admin runs the algo to generate a timetable for the students.

- Manage admins:

Admin can add admins, see the name, id, role and the department of every administrator. and view the actions that were set.

- Manage instructor:

Admin can add instructors, see the name, id, instructor id and view the actions that were set on every student.

- Manage Student:

Admin can enroll students, see the name, id, the status (spring, summer, fall) and view the actions that were set on every student.

Student:

- Login:

Student can login to the system and interact with our features by typing the student email and the password.

- View Dashboard:

Student can view enrolled courses, his type of semester, his timetable and the number of instructors teaching him.

- View Timetable:

Student can view his time table along the semester.

- Manage enrollments:

Student can view his data and can choose between courses and the courses has data like (course name, number of professors in every course and number of students in every course).

Instructor:

- Login:

instructor can login to the system and interact with our features by typing the instructor email and the password.

- View dashboard:

instructor can view the courses he participates in teaching, his total number of students he is already teaching, active features and average class size.

- View timetable:

instructor can view his time table along the semester.

Non-functional requirements

- Ensuring System Security s Access Control:

Since your project involves three distinct roles, maintaining strict boundaries and data integrity is crucial.

- Authenticating Users Securely:

Validating user credentials (Admin, Instructor, Student) via secure protocols before granting access to the system.

- Authorizing Role-Based Access:

Restricting system capabilities based on roles ensuring students can only view, instructors can input preferences, and only admins can generate or modify the master timetable.

- Optimizing Performance s Efficiency:

Timetable generation (especially using algorithms like Genetic Algorithms or Simulated Annealing) is computationally heavy. Your system must handle this load smoothly.

- Minimizing Response Time:

Delivering requested timetables to students and instructors in less than 2 seconds under normal network conditions.

- Accelerating Timetable Generation:

Processing and generating the optimal conflict-free schedule by the Admin within an acceptable time limit (e.g., under 5 minutes for a standard department dataset).

- Handling Concurrent Requests:

Managing high traffic volumes seamlessly when hundreds of students and instructors log in simultaneously to check schedules during peak release hours.

- Maximizing Availability s Reliability:

A scheduling system is mission-critical for an educational institution; downtime can disrupt an entire semester.

- Maintaining System Uptime:

Guaranteeing that the platform remains operational and accessible to all users at least 99.5% of the time during the academic semester.

- Recovering From Failures:

Backing up timetable configurations automatically so the system can recover to its last stable state within minutes if a server failure occurs.

- Preventing Data Conflicts:

Ensuring transactional integrity so that two simultaneous admin modifications do not corrupt the schedule or create ghost room allocations.

- Improving Usability s Accessibility:

Simplifying User Interface: Creating a clean, intuitive layout so Admins can easily input constraints (rooms, timeslots, course loads) and students can view their schedules at a glance without a learning curve.

- Adapting To Mobile Screens:

Ensuring a responsive web design that formats the complex timetable grid flawlessly on mobile devices, as students and instructors frequently check schedules on the go.

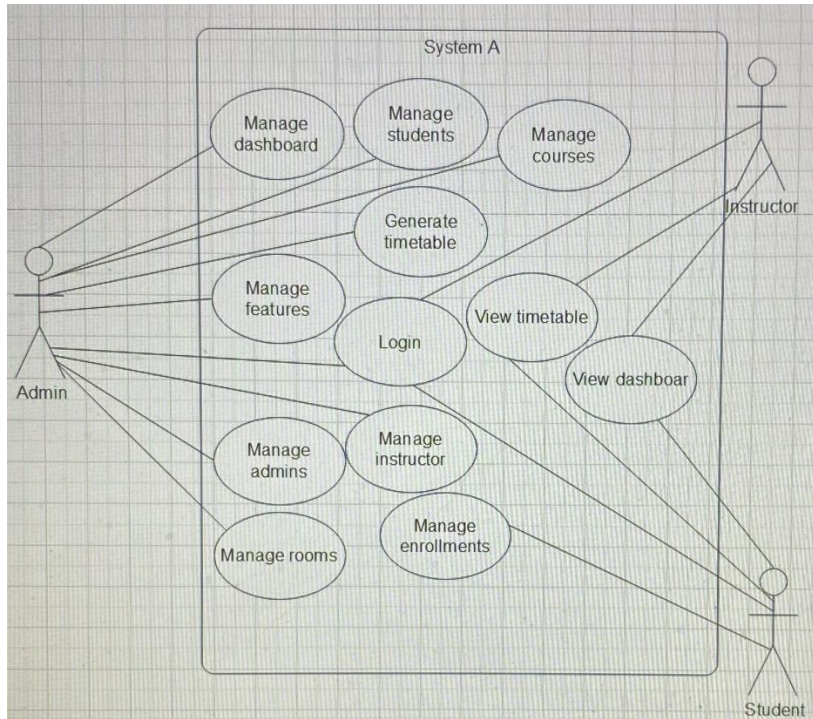
- Facilitating Maintainability s Scalability:

Scaling System Capacity: Accommodating growth in the number of students, courses, instructors, and classrooms in future semesters without degrading performance or requiring architectural changes.

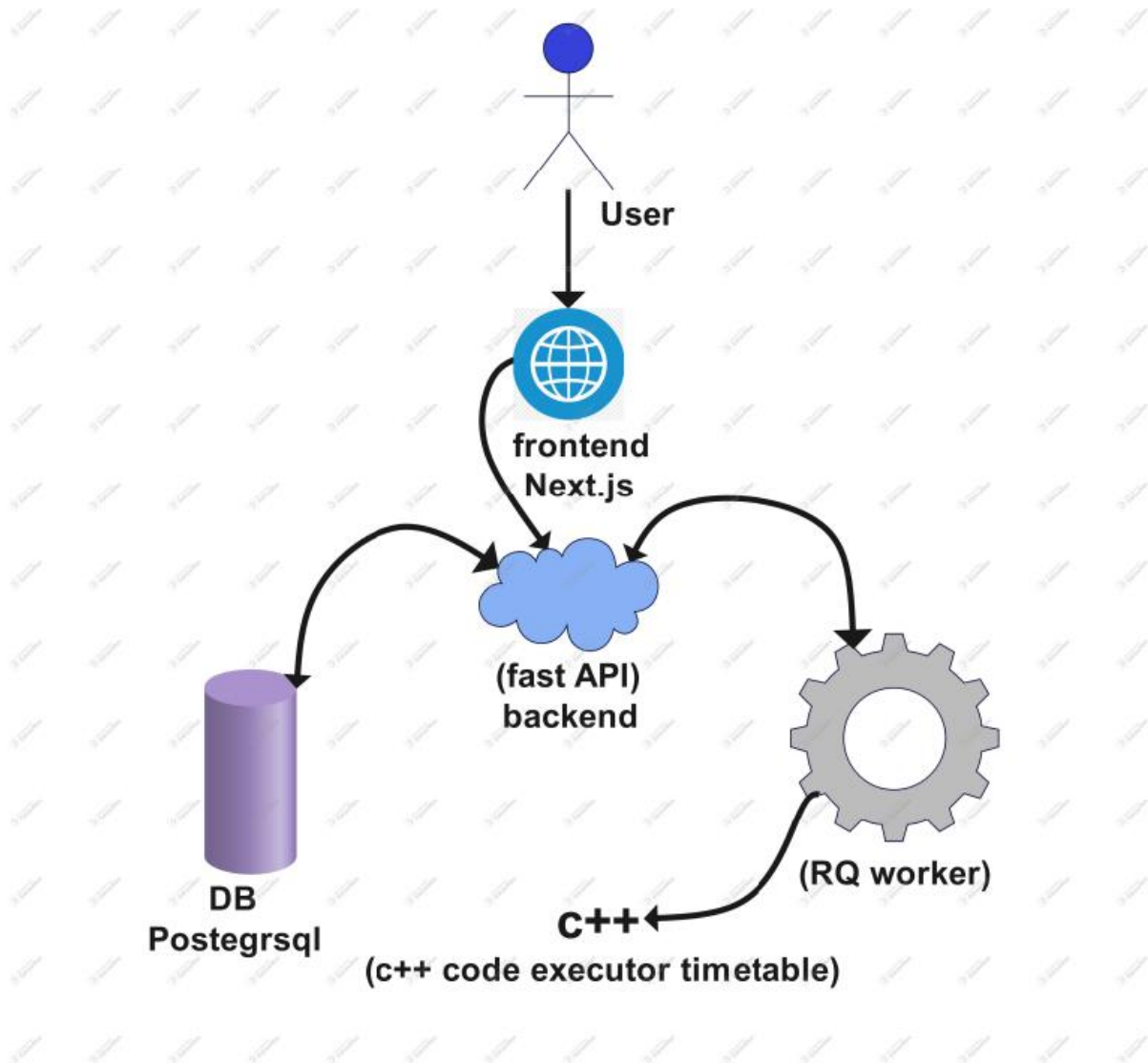
- Logging System Events:

Auditing system activities (e.g., logging exactly when an Admin overrides a schedule or changes a room constraint) to assist in debugging and tracking modifications.

Use-Case Diagram



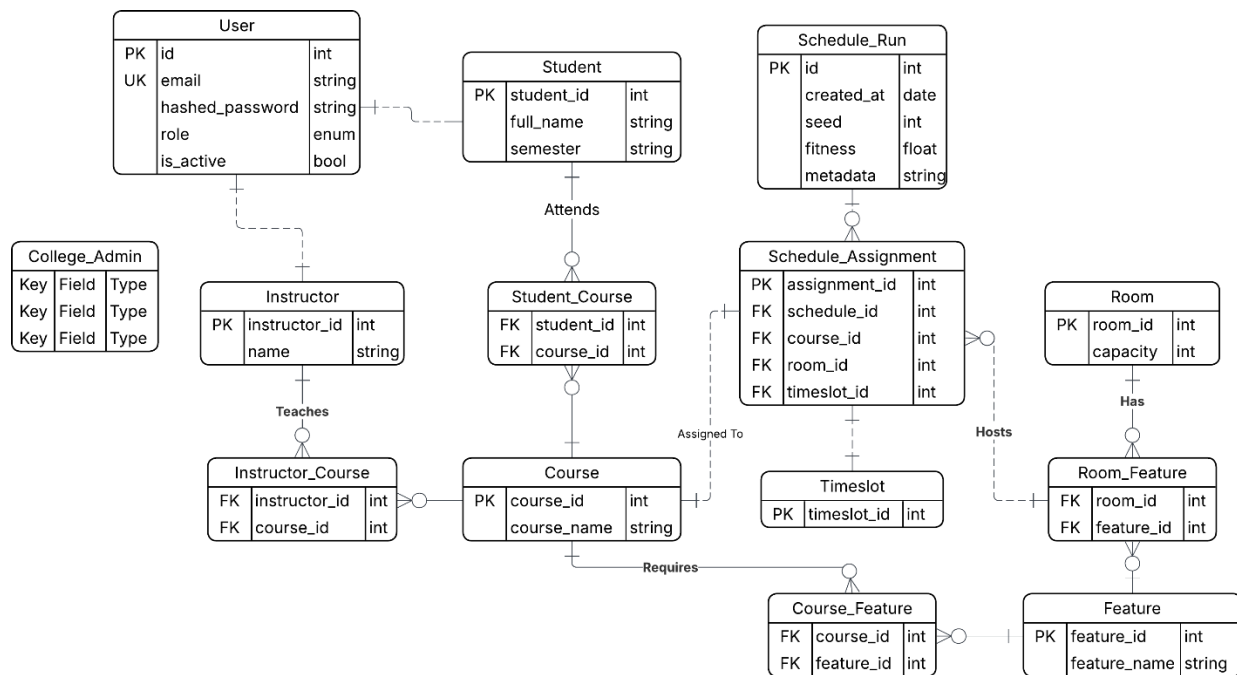
System Design



- **Presentation Layer (Frontend):** Developed using Next.js, this layer provides a modern, responsive user interface. It acts as the primary point of interaction for the user, collecting configuration parameters, managing institutional data inputs, and displaying the generated schedules.
- **Application Layer (Backend):** A FastAPI service serves as the central orchestrator. It exposes RESTful endpoints, handles data validation, manages database transactions, and delegates heavy processing tasks to the asynchronous queue rather than executing them on the main thread.
- **Data Persistence (Database):** A PostgreSQL relational database is utilized to securely store structured institutional data, including events, rooms, student enrollments, constraints, and the finalized timetables.

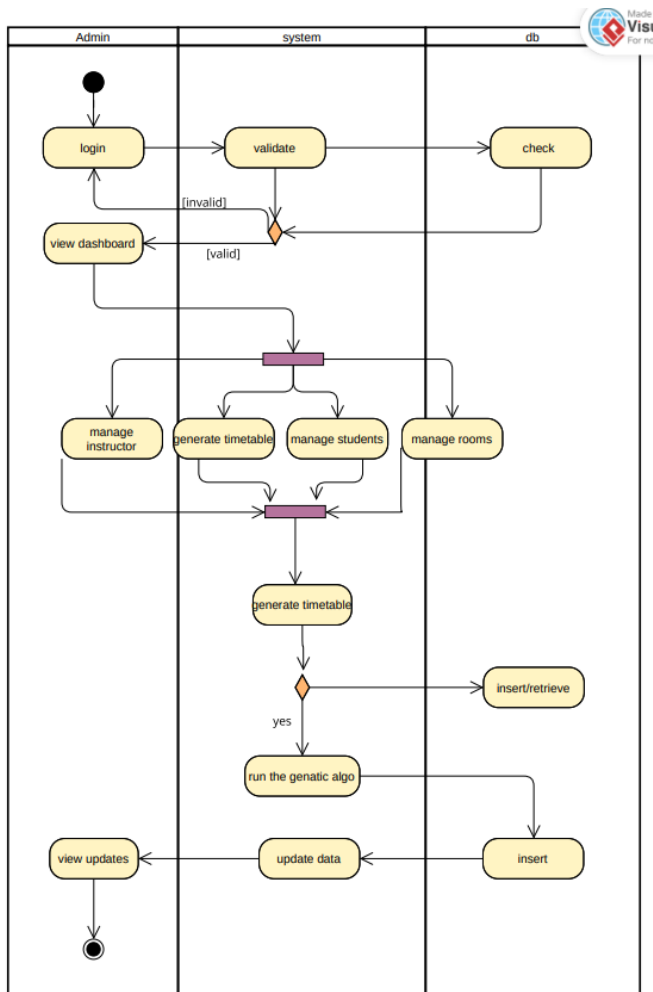
- **Asynchronous Task Management (RQ Worker):** To prevent the rigorous optimization process from blocking the API, an RQ (Redis Queue) worker is implemented. The FastAPI backend pushes scheduling jobs to this queue, which the worker then processes asynchronously in the background.
- **Optimization Engine (C++ Executor):** The core timetabling solver is implemented as an independent C++ executable. Triggered by the RQ worker, this high-performance engine performs the rigorous evolutionary computations required to solve the NP-hard timetabling constraints. Once execution is complete, the optimal timetable is returned to the backend for database storage and client retrieval.

ERD

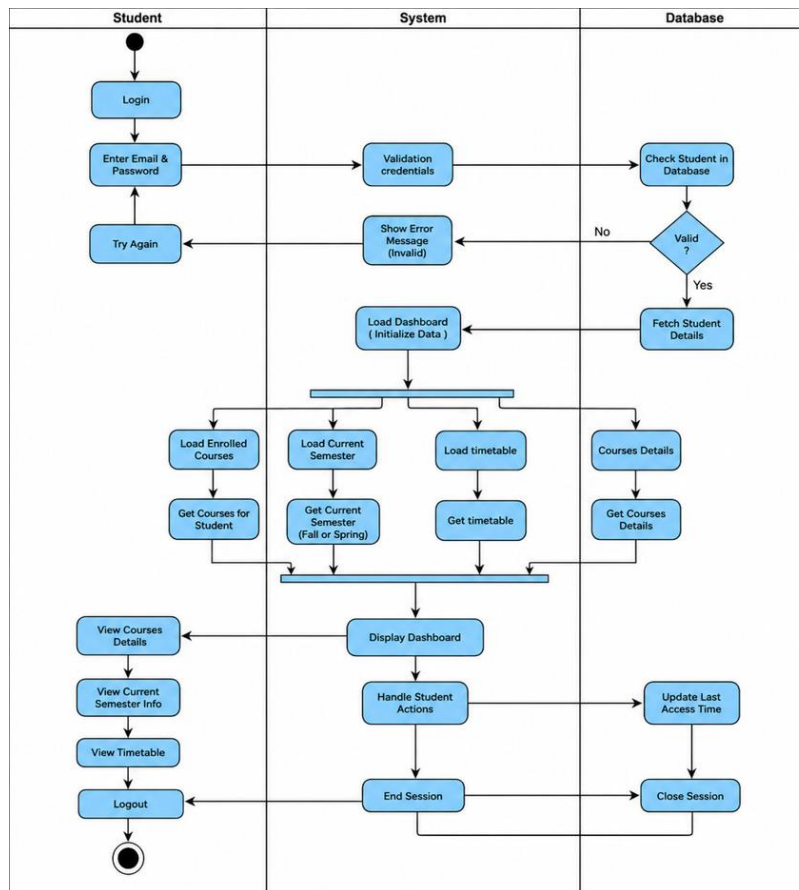


Activity Diagrams

Admin



Students



Chapter 8: Conclusion and Future Work

Conclusion

The University Course Timetabling Problem (UCTP) and its more restrictive variant, the Post Enrolment Course Timetabling Problem (PECTP), represent classically complex, NP-hard combinatorial challenges. The inherent difficulty of these problems lies in the massive search spaces and the necessity to simultaneously satisfy strict operational hard constraints while minimizing desirable soft constraints.

This thesis successfully addressed these challenges by developing a novel, high-performance educational management framework that bridges the gap between theoretical operational research and practical administrative software engineering. Rather than relying on a single optimization technique, this research designed and implemented a robust C++ algorithmic engine encompassing a suite of advanced

metaheuristics: Memetic Algorithms (MA), Cultural Algorithms (CA), Guided Search NSGA-II (GSNSGA-II), and a Hybrid Genetic Algorithm with Tabu Search (HGATS).

Through rigorous experimental evaluation against the internationally recognized ITC-2002 and ITC-2007 benchmark datasets, the proposed solvers demonstrated exceptional robustness. The integration of data-oriented architectural optimizations, such as constant-time delta evaluation and adaptive search mechanisms, allowed the engine to consistently achieve 100% feasibility (zero hard constraint violations) across diverse instances while aggressively minimizing soft constraint penalties. Furthermore, by transitioning the evaluation paradigm from a scalar penalty model to a Multi-Objective Optimization Problem (MOOP), the GSNSGA-II implementation successfully mapped the Pareto-optimal front, offering distinct trade-offs between competing timetabling objectives.

To ensure these theoretical achievements translate into a usable institutional tool, the optimization engine was embedded within a modern, decoupled web architecture. By utilizing a Next.js frontend for responsive user interaction, a FastAPI backend for RESTful orchestration, a PostgreSQL database for reliable data persistence, and a Redis-based task queue (RQ Worker) for asynchronous execution, the framework successfully abstracts the immense computational complexity of evolutionary algorithms away from the end-user. The resulting system serves as a highly scalable, viable model for modern university timetabling.

Future Work

While this research provides a comprehensive and highly optimized solution for the PECTP, the modular nature of the system architecture and the underlying C++ engine present numerous avenues for future enhancement. Future work will focus on expanding the mathematical constraint models, scaling the computational architecture, and deepening institutional integration.

Algorithmic and Constraint Expansion

- **Explicit Lecturer Modeling:** To maintain alignment with the ITC benchmark datasets, this project abstracted the lecturer entity into event-conflict matrices. Future iterations should expand the relational database and the mathematical model to explicitly include lecturer entities, enabling the optimization of highly granular, individualized faculty availability matrices and pedagogical preferences.

- **Curriculum-Based Course Timetabling (CBCTP):** The current algorithms are heavily optimized for student enrollment data (PECTP). Expanding the solver's logic to natively handle CBCTP tracks where conflicts are driven strictly by predefined university curricula would broaden the framework's applicability to different institutional models.

Architectural and System Scaling

- **Distributed Processing Cluster:** The current asynchronous architecture utilizes a single-node RQ worker to process the C++ engine. To effectively scale the optimization engine for massive, university-wide datasets encompassing tens of thousands of students, the architecture should be upgraded to support distributed processing across a multi-node computing cluster.
- **Advanced User Analytics:** Expanding the Next.js frontend to include advanced data visualization dashboards would provide university administrators with deeper analytical insights into room utilization rates, departmental bottlenecks, and long-term resource allocation trends.